

SISTEM REKOMENDASI FILM MENGGUNAKAN DATA USER-END DAN KNOWLEDGE GRAPH CONVOLUTIONAL NETWORK

SKRIPSI

Diajukan untuk Memenuhi Salah Satu Syarat
Matakuliah Tugas Akhir 2
Jenjang Strata 1 pada Program Studi Informatika
Universitas Jenderal Achmad Yani

Oleh:

Nama : Muhammad Rizki Yanuar

NIM : 3411211062



PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS SAINS DAN INFORMATIKA
UNIVERSITAS JENDERAL ACHMAD YANI

2025

HALAMAN PENGESAHAN
SISTEM REKOMENDASI FILM MENGGUNAKAN DATA USER-END
DAN KNOWLEDGE GRAPH CONVOLUTIONAL NETWORK

Oleh
Muhammad Rizki Yanuar
NIM.3411211062

Setelah membaca tugas akhir (skripsi/tesis) dengan seksama, menurut pertimbangan kami telah memenuhi persyaratan ilmiah sebagai suatu tugas akhir (skripsi/tesis)

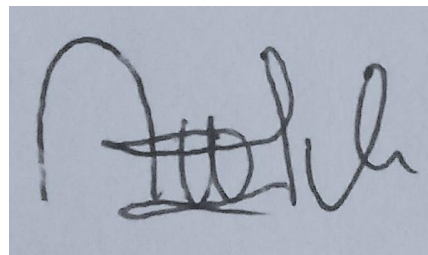
Cimahi,
Menyetujui

Pembimbing 1



Fajri Rakhmat Umbara, S.T., M.T.
NID. 412185888

Pembimbing 2



Agus Komarudin, S.Kom, M.T.
NID. 412175878

Mengetahui,

Dekan
Fakultas Sains dan Informatika

Ketua Program Studi Teknik
Informatika

Dr. Anceu Murniati, S.Si., M.Si.
NID. 4121 263 69

Asep Id H., S.Si., M.Kom., Ph.D.
NID. 4121 800 78



LEMBAR PERNYATAAN KEASLIAN PENULISAN SKRIPSI

Saya yang bertandatangan dibawah ini, dengan ini menyatakan bahwa penulisan skripsi yang telah saya susun dengan judul seperti berikut:

**“SISTEM REKOMENDASI FILM MENGGUNAKAN DATA USER-END
DAN KNOWLEDGE GRAPH CONVOLUTIONAL NETWORK”**

Merupakan hasil karya saya sendiri. Berkas beserta program yang telah dibuat merupakan hasil pekerjaan saya sepenuhnya. Ide, pendapat atau materi yang berasal dari sumber lain telah dikutip dengan penulisan referensi yang sesuai dengan aturan dan bersifat baku.

Demikian pernyataan ini telah saya buat.

Cimahi, 2025

Muhammad Rizki Yanuar

NIM. 3411 21 1062

ABSTRAK

Penelitian ini bertujuan untuk mengatasi kelemahan sistem rekomendasi tradisional seperti Collaborative Filtering dan Content-Based Filtering, yang seringkali gagal menyajikan rekomendasi yang terpersonalisasi akibat masalah sparsity, cold-start. Untuk menjawab tantangan tersebut, diusulkan pendekatan berbasis Knowledge Graph Convolutional Network (KGCN) yang memanfaatkan struktur Knowledge Graph (KG) dari dataset MovieLens 1M, yang diperkaya dengan data demografis pengguna. Fokus utama eksperimen ini adalah membandingkan efektivitas metode Importance Sampling (IS)—yang memilih tetangga berdasarkan bobot relevansinya—dengan pendekatan dasar Uniform Sampling (US) sebagai solusi atas tingginya kompleksitas komputasi pada GCN. Hasil pengujian akhir pada data tes secara konklusif menunjukkan bahwa model yang dioptimalkan dengan IS secara signifikan mengungguli US di semua metrik, dengan peningkatan paling signifikan terlihat pada Precision@10 (0.4988 untuk IS dibandingkan 0.2845 untuk US). Selain itu, proses hyperparameter tuning menemukan bahwa arsitektur model yang lebih ramping (embedding size 32) tidak hanya memberikan kinerja generalisasi terbaik, tetapi juga meningkatkan efisiensi waktu pelatihan secara signifikan. Secara keseluruhan, penelitian ini membuktikan bahwa integrasi knowledge graph dengan importance sampling merupakan pendekatan yang efektif dan efisien untuk membangun sistem rekomendasi film yang lebih akurat serta andal.

Kata Kunci: Knowledge Graph, Graph Convolutional Network, Sistem Rekomendasi Film, Cold Start, Sparsity, Data User-End

ABSTRACT

This research aims to address the weaknesses of traditional recommendation systems such as Collaborative Filtering and Content-Based Filtering, which often fail to provide personalised recommendations due to issues of sparsity, cold-start. To address these challenges, a Knowledge Graph Convolutional Network (KGCN)-based approach is proposed, leveraging the Knowledge Graph (KG) structure from the MovieLens 1M dataset, enriched with user demographic data. The main focus of this experiment is to compare the effectiveness of the Importance Sampling (IS) technique—which selects neighbours based on their relevance weights—with the basic Uniform Sampling (US) approach as a solution to the high computational complexity of GCN. Final testing on the test data conclusively shows that the model optimised with IS significantly outperforms US across all metrics, with the most significant improvement observed in Precision@10 (0.4988 for IS compared to 0.2845 for US). Additionally, the hyperparameter tuning process found that a leaner model architecture (embedding size 32) not only provides the best generalisation performance but also significantly improves training time efficiency. Overall, this research demonstrates that integrating knowledge graphs with importance sampling is an effective and efficient approach for building more accurate and reliable film recommendation systems.

Keyword: Knowledge Graph, Graph Convolutional Network, Movie Recommendation System, Cold-Start, Sparsity, Data User-End

KATA PENGANTAR

Puji dan syukur saya panjatkan ke hadirat Tuhan Yang Maha Esa, karena atas rahmat dan karunia-Nya, saya dapat menyusun proposal Tugas Akhir ini sebagai salah satu syarat akademik dalam menyelesaikan studi di program ini. Proposal ini disusun sebagai langkah awal dalam penelitian yang akan saya lakukan guna menyelesaikan Tugas Akhir. Dalam proses penyusunan proposal ini, saya memperoleh banyak wawasan dan pemahaman baru dari berbagai referensi serta bimbingan dari dosen pembimbing. Oleh karena itu, saya ingin mengucapkan terima kasih yang sebesar-besarnya kepada dosen pembimbing yang telah memberikan arahan dan masukan yang sangat berharga. Tak lupa, saya juga menyampaikan apresiasi kepada teman-teman dan keluarga yang selalu memberikan dukungan moral dan motivasi dalam menyelesaikan tahap awal penelitian ini. Saya menyadari bahwa dalam penyusunan proposal ini masih terdapat banyak kekurangan. Oleh karena itu, saya sangat mengharapkan saran dan kritik yang membangun agar penelitian ini dapat berkembang lebih baik dan memberikan kontribusi yang berarti. Semoga proposal ini dapat menjadi landasan yang kuat dalam pelaksanaan penelitian Tugas Akhir dan bermanfaat bagi pengembangan ilmu pengetahuan di bidang yang saya tekuni. Demikian pengantar ini saya sampaikan. Semoga penelitian ini dapat berjalan dengan lancar dan memberikan hasil yang maksimal.

Hormat Kami,

Muhammad Rizki Yanuar

3411211062

Cimahi, 2025

DAFTAR ISI

HALAMAN PENGESAHAN.....	i
LEMBAR PERNYATAAN KEASLIAN PENULISAN SKRIPSI	ii
ABSTRAK	iii
KATA PENGANTAR.....	v
DAFTAR ISI	vi
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	4
1.3 Batasan Masalah.....	4
1.4 Tujuan Penelitian.....	4
1.5 Metodologi Penelitian	5
1.5.1 Teknik Pengumpulan Data	5
1.5.2 Teknik Penerapan Metode atau konsep.....	5
1.6 Sistematika Penulisan	5
BAB II TINJAUAN PUSTAKA	7
2.1 Data Mining	7
2.2 Sistem Rekomendasi	7
2.2.1 Collaborative Filtering (CF).....	7
2.2.2 Kelebihan dan Kekurangan Collaborative Filtering	8
2.2.3 Content-Based Filtering	8
2.2.4 Kelebihan dan Kekurangan Content-Based Filtering	9
2.2.5 Hybrid Filtering.....	9

2.3	Knowledge Graph Convolutional Network	10
2.3.1	Knowledge Graph	10
2.3.2	Graph Convolutional Network.....	12
2.3.3	Message Passing dan Sum Aggregation.....	12
2.3.4	Skor Prediksi	13
2.4	Importance sampling (IS).....	13
2.5	Hyperparameter Tuning	14
2.6	Metrik Evaluasi	15
2.7	Output.....	16
BAB III	METODOLOGI PENELITIAN.....	18
3.1	Pendekatan Penelitian	18
3.2	Metodologi Penelitian	19
3.3	Dataset.....	19
3.4	Preprocessing Data.....	21
3.4.1	Penggabungan Dataset	21
3.4.2	Label Encoding	21
3.5	Implementasi Knowledge Graph.....	22
3.5.1	Representasi Node.....	22
3.5.2	Data Interaksi User-Item	23
3.5.3	Triplet Hubungan Entitas Relasi Entitas	23
3.6	Splitting Data	23
3.7	Implementasi Graph Convolutional Network	23
3.7.1	Embedding Awal	24
3.7.2	Importance Sampling	25
3.7.3	Message Passing dan Sum Aggregation.....	27

3.7.4	Prediksi.....	28
3.8	Perhitungan Metrik Evaluasi.....	29
3.8.1	Recall@10.....	29
3.8.2	Precision@10	29
3.8.3	NDCG@K.....	29
3.8.4	AUC	31
BAB IV	HASIL DAN PEMBAHASAN	34
4.1	Hasil Preprocessing Data dan Pembentukan Graf	34
4.1.1	Dataset.....	34
4.1.2	Penggabungan Dataset	35
4.1.3	Label Encoding	36
4.1.4	Inisialisasi Knowledge Graph	36
4.1.5	Splitting Dataset.....	38
4.2	Inisialisasi Model Graph Convolutional Network	38
4.3	Pelatihan Model	38
4.4	Hyperparameter.....	40
4.5	Pelatihan Ulang	42
4.6	Test Model.....	43
4.7	Analisis dan Studi Kasus: Perbandingan Pengguna Baru vs Pengguna Lama	43
4.7.1	Studi Kasus Pengguna Baru	43
4.7.2	Studi Kasus: Pengguna Lama	44
BAB V	KESIMPULAN DAN SARAN.....	46
5.1	Kesimpulan	46
5.2	Saran.....	46

DAFTAR PUSTAKA	48
LAMPIRAN	52

DAFTAR GAMBAR

Gambar III. 1 Metode Penelitian.....	19
Gambar III. 2 Arsitektur Knowledge Graph	22
Gambar III. 3 Arsitektur Graph Convolutional Network	24
Gambar IV. 1 Data Film	34
Gambar IV. 2 Data Rating	35
Gambar IV. 3 Data Pengguna	35
Gambar IV. 4 Penggabungan Dataset	36
Gambar IV. 5 Label Encoding	36
Gambar IV. 6 Metrik Evaluasi	40
Gambar IV. 7 Pelatihan Ulang Importance Sampling	42
Gambar IV. 8 Pelatihan Uniform Sampling	42
Gambar IV. 9 Hasil Rekomendasi Pengguna Baru	44
Gambar IV. 10 Hasil Rekomendasi Pengguna Lama	45

DAFTAR TABEL

Tabel II. 1 Hyperparameter Tuning	15
Tabel III. 1 Atribut Pengguna	20
Tabel III. 2 Atribut Film	20
Tabel III. 3 Data Rating	21
Tabel III. 4 Triplet Hubungan Entitas Relasi Entitas	23
Tabel III. 5 Embedding Awal	25
Tabel III. 6 Evaluasi DCG@K	29
Tabel III. 7 Evaluasi IDCG@K	30
Tabel III. 8 Evaluasi AUC	31
Tabel IV. 1 Trilplet Hubungan	37
Tabel IV. 2 Parameter Awal Model	38
Tabel IV. 3 Perbandingan Waktu Komputasi Training	39
Tabel IV. 4 Hyperparameter Model	41
Tabel IV. 5 Hasil Evaluasi Data Tes	43

BAB I PENDAHULUAN

1.1 Latar Belakang

Sistem rekomendasi film adalah perangkat lunak pintar yang menyarankan film kepada pengguna berdasarkan minat, riwayat tontonan, dan informasi relevan lainnya. Untuk membuat saran yang dipersonalisasi, sistem ini menganalisis kumpulan data yang sangat besar, termasuk peringkat, metadata film, dan pola perilaku pengguna. Tujuan utamanya adalah meningkatkan pengalaman menonton dengan memberikan rekomendasi khusus yang sesuai selera, sehingga pengguna tidak perlu menelusuri koleksi film yang sangat banyak secara manual [1]. Di dalam cara kerja sistem rekomendasi yang rumit, terdapat beragam metode yang bertujuan untuk mengungkap pola dan kesamaan antara pengguna dan item. Secara luas metode ini diklasifikasikan ke dalam dua pendekatan utama, yaitu *content-based filtering* yang menganalisis atribut item seperti genre dan sutradara, serta *collaborative filtering* yang mempelajari perilaku interaksi para pengguna [2]. Meskipun telah digunakan secara luas, metode tradisional seperti *collaborative filtering* dan *content-based filtering* sering kali menghadapi masalah *sparsity* dan *cold start* [3] [2].

Untuk mengatasi masalah ini, penelitian telah beralih ke pendekatan *deep learning*, termasuk penggunaan *Knowledge Graph Convolutional Networks* (KGCN). *Knowledge Graph* (KG) merupakan sebuah grafik yang menyimpan informasi terarah dengan *node* yang mewakili entitas (seperti pengguna, item atau atribut) dan tepi yang mewakili hubungan antar entitas. Salah satu keunggulan KG adalah kemampuan dalam menghubungkan berbagai atribut dan informasi yang relevan untuk pengguna dan item. *Graph Convolutional Network* (GCN) merupakan metode yang dirancang untuk bekerja dengan data yang terstruktur dalam bentuk graf. Pada GCN terdapat fitur utama yaitu kemampuan dalam mengagregasi dan menggabungkan informasi dari tetangga terdekat, desain semacam ini memiliki 2 keuntungan: (1) Struktur kedekatan lokal secara efektif dicatat dan disimpan di setiap entitas melalui metode agregasi. (2) Tetangga diberi bobot berdasarkan skor yang bergantung pada pengguna tertentu dan hubungan

antar entitas. Skor ini menggambarkan informasi konten dari *Knowledge Graph* (KG) serta minat relasional pengguna itu sendiri [4]. Meskipun demikian, penerapan *Knowledge Graph* (KG) pada sistem rekomendasi memiliki beberapa tantangan seperti masalah dimensi yang tinggi dan heterogenitas yang disebabkan adanya berbagai macam entitas dan hubungan yang terbentuk [4]. Tantangan semakin kompleks saat diimplementasikan pada *Graph Convolutional Network* (GCN) berskala besar, dimana model GCN umumnya terkendala oleh tingginya kompleksitas waktu dan ukuran memori akibat besarnya jumlah *node* serta interaksi. Untuk mengatasi masalah tersebut, diterapkan metode sampling pada graph heterogenitas, dimana teknik sampling ini akan mengambil nodes yang lebih kecil pada *layer* tertentu tetapi representatif dari distribusi yang memberikan bobot lebih pada *layer* tertentu [5]. Berbagai penelitian telah dilakukan dalam pengembangan sistem rekomendasi yang memanfaatkan data item-end dan user-item dengan menggunakan berbagai metode.

Pada penelitian [4] penggunaan KGCN dengan memanfaatkan data item-ends dan menggunakan *uniformly sampling fixed-size neighborhood* untuk mengatasi jumlah nodes tetangga yang sangat besar, yang menghasilkan rekomendasi berdasarkan atribut-atribut item dan mengurangi waktu komputasi karena menentukan jumlah nodes tetangga yang akan diambil. Penelitian ini menunjukkan bahwa KGCN dapat efektif dalam mengidentifikasi keterkaitan antar item dari atribut yang terdapat dalam *Knowledge Graph* dan *uniformly sampling fixed-size neighborhood* menunjukkan peningkatan rata-rata AUC 4,4% karena dapat menangkap informasi yang jauh lebih banyak. Meskipun penelitian ini memberikan hasil signifikan terhadap pengembangan sistem rekomendasi berbasis item, masih terdapat celah dalam pemanfaatan data user-end dan teknik sampling. Penelitian lebih lanjut diperlukan untuk mengeksplorasi bagaimana umpan balik pengguna, baik bersifat eksplisit maupun implisit dapat diintegrasikan ke dalam model KGCN dan bagaimana teknik *importance sampling* dapat meningkatkan akurasi model, dengan demikian sistem rekomendasi tidak hanya bergantung pada atribut item tetapi juga mempertimbangkan preferensi dan pengalaman pengguna serta menangkap informasi yang jauh lebih relevan ketika pemilihan nodes tetangga.

Pada penelitian [6] penggunaan sistem rekomendasi hybrid yang menggabungkan *Collaborative Filtering*, *Content-Based Filtering*, dan teknik *Self-Organizing* (SOM) dengan memanfaatkan data user-item sebagai dasar untuk mengembangkan model rekomendasi. Kemudian pendekatan hybrid ini memanfaatkan metode *weighted* dengan menggabungkan skor dari collaborative filtering dan content-based filtering untuk menghasilkan satu rekomendasi dan *Feature Augmentation* dengan memanfaatkan output dari hasil rekomendasi *collaborative dan content-based* kemudian menjadikan hasil rekomendasi tersebut sebagai informasi tambahan dalam menciptakan model rekomendasi. Penelitian ini menyarankan penerapan *Deep Learning* seperti *Graph Neural Network* (GNN) untuk menangkap pola interaksi yang lebih kompleks, pengembangan strategi untuk menangani masalah cold start bagi pengguna baru atau item baru, dan mengintegrasikan data user-end untuk mengeksplorasi bagaimana umpan balik eksplisit dan implisit dari pengguna dapat meningkatkan akurasi rekomendasi.

Pada penelitian [7], pengaruh knowledge graph memberikan hasil AUC 0.982 dijelaskan melalui penerapan metode CUIKG (Combining User-end and Item-end Knowledge Graph learning). Metode ini secara bersamaan mempelajari embedding pengguna dan item dari knowledge graph di kedua sisi, yang memungkinkan model untuk menangkap relevansi antara pengguna dan item dengan lebih baik. Dengan memanfaatkan informasi semantik yang kaya dari knowledge graph, model ini dapat mengatasi masalah sparsity data dengan memperkenalkan lebih banyak hubungan semantik yang mendukung akurasi rekomendasi. Selain itu, model ini memperhitungkan atribut pengguna dan bagaimana atribut tersebut mempengaruhi preferensi mereka terhadap item, sehingga menciptakan representasi yang lebih akurat. Tetapi penelitian ini memiliki keterbatasan dimana model rentan terhadap *over-smoothing* dimana performa model justru akan menurun jika lapisan GCN yang digunakan terlalu banyak, sehingga jumlah lapisan terbaik berada di rentang 1-2 lapisan.

Beberapa penelitian sebelumnya telah menerapkan KGCN untuk sistem rekomendasi, namun penggunaan data *user-end* dan teknik *importance sampling*

belum banyak diterapkan pada sistem rekomendasi. Mayoritas penelitian yang menggunakan KGCN cenderung menggunakan data interaksi antara pengguna-item, dan hanya menerapkan teknik *uniform sampling*. Penelitian ini berfokus pada mengatasi tantangan utama dalam sistem rekomendasi film, seperti masalah sparsity, cold-start, dan efisiensi komputasi. Dengan mengintegrasikan data user-end, termasuk umpan balik eksplisit dan implisit, ke dalam Knowledge Graph Convolutional Network (KGCN), sistem rekomendasi dapat menangkap preferensi pengguna secara lebih personalisasi, tidak hanya bergantung pada atribut item. Selain itu, teknik *importance sampling* diterapkan untuk mengoptimalkan pemilihan nodes tetangga dalam Knowledge Graph, memastikan hanya informasi yang relevan diambil, sehingga mengurangi beban komputasi tanpa mengorbankan akurasi rekomendasi. Performa model yang diusulkan akan dievaluasi dan dibandingkan dengan teknik *uniform sampling* mengukur peningkatan dalam akurasi, relevansi rekomendasi, dan efisiensi waktu komputasi.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah dijelaskan, rumusan masalah dalam penelitian ini yaitu, bagaimana hasil perbandingan penerapan *importance sampling* dan *uniform sampling* yang telah diperkaya dengan data *user-end* dalam menangani masalah sparsity dan cold start, yang ditinjau dari efisiensi waktu komputasi selama pelatihan dan tingkat akurasi yang diukur dengan Precision@K, Recall@K, NDCG@K, dan AUC?

1.3 Batasan Masalah

Penelitian ini hanya menggunakan dataset dari platform MovieLens 1M tanpa menggabungkan data diluar platform tersebut, fokus perbandingan model dalam penelitian ini hanya terbatas pada dua teknik *sampling* dalam KGCN, yaitu *importance sampling* dan *uniform sampling*.

1.4 Tujuan Penelitian

Tujuan penelitian ini adalah untuk membandingkan pelatihan model KGCN yang menggunakan teknik *importance sampling* dan *uniform sampling* yang telah ditambahkan dengan data user-end. Penelitian ini akan mengevaluasi kedua teknik

tersebut berdasarkan efisiensi waktu komputasi yang dibutuhkan selama proses pelatihan, serta kinerja model yang diukur menggunakan metrik evaluasi Precision@K, Recall@K, NDCG@K, AUC.

1.5 Metodologi Penelitian

1.5.1 Teknik Pengumpulan Data

Pengambilan data diambil dari website movielens, dimana data yang diambil memiliki 100.000 data rating. Dataset terbagi menjadi tiga bagian:

- Data Pengguna: Berisi demografi user,
- Data film: Berisi genre untuk film
- Data rating: Berisikan skala rating 1-5 yang diberikan kepada film yang ditonton.

1.5.2 Teknik Penerapan Metode atau konsep

Metode yang digunakan berupa label encoding dalam merubah data yang bersifat kategorik menjadi bentuk numerik, selanjutnya dari tiga data yang terpisah akan dilakukan penggabungan dataset untuk mendapatkan informasi yang lebih jelas.

Setelah itu dilakukan pembuatan *knowledge graph* untuk membentuk graph hubungan antar node kemudian akan dilakukan splitting data menjadi 3 yaitu *data train*, *data validation* dan *data test* dengan pembagian 80:10:10.

Setelah dilakukan splitting data, pada model *graph convolutional network* dibuat nilai embedding awal pada masing-masing *nodes*, kemudian diterapkan *importance sampling* untuk menentukan mana nodes yang memiliki informasi paling relevan berdasarkan bobot relasi antar node, kemudian embedding node pusat akan diperbarui menggunakan *message passing* dimana informasi dari node tetangga akan dijumlahkan dengan informasi node pusat menggunakan *sum aggregation*, setelah itu akan dilakukan prediksi terhadap hubungan kepada node item.

1.6 Sistematika Penulisan

Sistematika penulisan yang digunakan dalam penulisan laporan tugas akhir ini yaitu sebagai berikut:

BAB 1

PENDAHULUAN

	BAB 1 menguraikan latar belakang, rumusan masalah, batasan masalah, tujuan penelitian, luaran dan manfaat penelitian, serta metodologi penelitian.
BAB 2	TINJAUAN PUSTAKA BAB 2 menguraikan definisi berbagai teori penunjang yang digunakan sebagai rujukan dalam penyusunan laporan.
BAB 3	ANALISIS DAN PERANCANGAN SISTEM BAB 3 berisi analisis dan perancangan sistem yang dikembangkan.
BAB 4	IMPLEMENTASI DAN PENGUJIAN SISTEM BAB 4 berisi penjelasan mengenai lingkungan implementasi, hasil dari setiap tahapan metodologi mulai dari preprocessing data, pembentukan knowledge graph, pelatihan model, hingga pengujian akhir.
BAB 5	KESIMPULAN DAN SARAN BAB 5 berisi kesimpulan dari keseluruhan hasil penelitian yang menjawab rumusan masalah, serta saran untuk pengembangan penelitian selanjutnya di masa mendatang.
DAFTAR PUSTAKA	Daftar pustaka berisi seluruh sumber referensi ilmiah, seperti jurnal, buku, dan prosiding, yang digunakan sebagai landasan teori dan rujukan dalam penelitian ini.
LAMPIRAN	Lampiran berisi dokumen-dokumen pendukung yang relevan dengan penelitian.

BAB II TINJAUAN PUSTAKA

2.1 Data Mining

Data mining adalah kumpulan metode yang mengekstrak informasi dari basis data yang besar, digunakan untuk menemukan korelasi, karakteristik, dan tren baru, dimana informasi tersebut harus membantu dalam pengambilan keputusan [8]. Proses ini dilakukan menggunakan teknologi statistik, ML dan pengenalan pola [9][10].

2.2 Sistem Rekomendasi

Sistem rekomendasi adalah alat penyaringan informasi yang menangani masalah dengan menyediakan konten yang menarik bagi pengguna dengan cara yang personalisasi [11]. Sistem rekomendasi merupakan teknologi yang berguna untuk meringankan masalah kelebihan informasi yang diberikan kepada pengguna, dimana sistem memberikan pengguna daftar item yang direkomendasikan yang mungkin mereka sukai atau memprediksi seberapa besar mereka menyukai setiap item [12].

2.2.1 Collaborative Filtering (CF)

Collaborative filtering adalah teknik sistem rekomendasi yang bekerja berdasarkan prinsip bahwa seseorang akan menyukai item yang juga disukai oleh orang lain dengan selera serupa. Secara umum, metode ini bekerja dengan mengumpulkan data interaksi dari banyak pengguna untuk membangun matriks item-pengguna. Berdasarkan data ini, sistem akan mencari sekelompok pengguna lain yang memiliki riwayat rating paling mirip, yang dikenal sebagai “lingkungan” atau *neighborhood*. Setelah kelompok ini terbentuk, sistem akan merekomendasikan item yang telah diberi rating tinggi oleh “tetangga” tersebut kepada target pengguna, khususnya pada item yang belum pernah dia nilai. Output yang dihasilkan dapat berupa prediksi skor (nilai numerik yang memprediksi rating pengguna terhadap suatu item), atau berupa daftar rekomendasi (N item teratas yang paling mungkin disukai pengguna) [13]. *Collaborative Filtering* mempunyai dua tipe algoritma yaitu *Memory-Based* dan *Model-Based*. *Memory-Based* menyimpan dan mengakses matrix interaksi untuk menghitung kemiripan antar

pengguna atau item untuk menemukan tetangga terdekat dan menghasilkan rekomendasi. *Model-Based* menggunakan matrik interaksi pengguna-item untuk melatih model seperti Singular Value Decomposition (SVD), regresi atau *clustering* dalam mengidentifikasi hubungan tersembunyi antar item [6].

2.2.2 Kelebihan dan Kekurangan Collaborative Filtering

Collaborative Filtering (CF) memiliki keunggulan utama. Pertama, dapat bekerja tanpa bergantung pada atribut item karena hanya mengandalkan pola rating dari banyak pengguna. Selain itu teknik CF dapat memberikan rekomendasi tak terduga, dimana pengguna bisa mendapatkan rekomendasi yang relevan hanya karena pengguna lain memiliki selera yang serupa. Namun, di balik keunggulan tersebut, *Collaborative Filtering* dihadapkan pada beberapa kekurangan mendasar, terutama saat diterapkan pada skenario dunia nyata dengan data berkala besar. Keterbatasan utama dari metode ini adalah masalah *Sparsity* dan *Cold-start*.

Sparsity merupakan masalah yang terjadi dikarenakan sedikitnya interaksi yang dilakukan pengguna terhadap item, sehingga membuat teknik ini tidak mampu menemukan tetangga yang serupa dan akhirnya menghasilkan rekomendasi yang lemah. Sedangkan *Cold-start* merupakan situasi dimana CF tidak memiliki informasi yang memadai mengenai pengguna ataupun item untuk membuat prediksi, hal ini terjadi karena adanya pengguna baru atau item baru yang belum pernah melakukan interaksi sama sekali [13].

2.2.3 Content-Based Filtering

Content-Based Filtering (CBF) adalah teknik sistem rekomendasi yang bergantung pada atribut item untuk dikelompokkan berdasarkan kemiripan atribut item. Berbeda dengan *collaborative filtering*, metode ini tidak memerlukan data dari pengguna lain. Dalam teknik CBF, rekomendasi dibuat berdasarkan profil preferensi pengguna, yang dibangun dari fitur atau atribut item yang pernah dinilai positif oleh pengguna di masa lalu. CBF kemudian akan merekomendasikan item baru yang memiliki atribut serupa berdasarkan profil preferensi. CBF menggunakan beberapa model untuk menemukan hubungan antar item, antara lain adalah *Term Frequence – Inverse Document Frequency* (TF-IDF) untuk analisis teks, serta model probabilitas seperti Naïve Bayes Classifier, Decision Tree [13].

2.2.4 Kelebihan dan Kekurangan Content-Based Filtering

Teknik CBF mampu mengatasi tantangan CF, dimana teknik ini memiliki kemampuan untuk merekomendasikan item baru meskipun tidak ada peringkat yang diberikan pengguna, dan jika preferensi user berubah, maka CBF mampu menyesuaikan hasil rekomendasi dalam waktu singkat.

Namun teknik CBF bergantung pada metadata item, oleh karena itu teknik CBF ini membutuhkan deskripsi item yang kaya sebelum rekomendasi dapat diberikan. Jika deskripsi item tidak lengkap atau tidak ada, maka kualitas rekomendasi akan menurun. Masalah lain yang terjadi adalah *content overspecialization* dimana sistem akan terus merekomendasikan item yang mirip dengan profil pengguna, sehingga akan membatasi penemuan minat baru [13].

2.2.5 Hybrid Filtering

Hybrid Filtering menggabungkan teknik rekomendasi untuk mendapatkan optimasi sistem yang lebih baik untuk menghindari keterbatasan dan masalah sistem rekomendasi. Menggunakan beberapa teknik rekomendasi dapat menekan kelemahan teknik model individu [13]. Temuan menunjukkan bahwa sebagian besar sistem rekomendasi yang ada saat ini menggunakan pendekatan *hybrid*, yang menggabungkan CF, CBF dan pendekatan lainnya. Pendekatan *hybrid* diimplementasikan dalam berbagai cara, seperti membuat prediksi menggunakan CF dan CBF secara terpisah, lalu menggabungkannya [14].

Melalui teknik ini menunjukkan bahwa *hybrid filtering* dapat meningkatkan kualitas rekomendasi dibandingkan dengan menggunakan satu metode saja. Pendekatan ini juga dapat menyelesaikan permasalahan *cold-start* yang terjadi dan juga memberikan keuntungan dalam memahami preferensi pengguna dengan lebih baik. Meskipun pendekatan *hybrid* memberikan hasil yang lebih baik, masih ada beberapa tantangan yang perlu dipertimbangkan, dimana pendekatan ini membutuhkan pemrosesan yang lebih kompleks dan biaya komputasi yang tinggi, serta *hybrid* model kesulitan dalam memodelkan hubungan yang lebih dalam antar entitas [14]. Untuk mengatasi keterbatasan tersebut pendekatan berbasis *Knowledge Graph* (KG) diperkenalkan ke dalam sistem rekomendasi yang

menunjukkan bahwa hubungan antara pengguna dan item serta preferensi pengguna dapat ditangkap lebih akurat dibandingkan dengan metode tradisional [3].

2.3 Knowledge Graph Convolutional Network

2.3.1 Knowledge Graph

Knowledge Graph (KG) diperkenalkan oleh Google pada tahun 2012 dengan tujuan untuk mencapai mesin pencari yang lebih cerdas, KG merupakan struktur data yang menyimpan informasi dalam bentuk entitas serta hubungan antar entitas [15]. Setiap node dalam KG merepresentasikan entitas, sedangkan edge menggambarkan jenis hubungan antara dua entitas. Dalam sistem rekomendasi, KG digunakan untuk mengumpulkan informasi tambahan dan konektivitas menjadi pengetahuan yang tidak hanya bergantung pada interaksi pengguna-item sehingga informasi menjadi lebih mudah dipahami sehingga membentuk informasi yang penuh makna [3]. Interaksi eksplisit antara pengguna dan item direpresentasikan dalam bentuk matrix interaksi:

$$Y \in \mathbb{R}^{M \times N} \quad (1)$$

Penjelasan:

Y = Interaksi antara pengguna dan item

$\mathbb{R}$ = Himpunan bilangan real dalam matrix Y

M = Jumlah baris yang mewakili jumlah pengguna dalam matrix

N = Jumlah kolom item yang mewakili jumlah item dalam matrix

Struktur KG membentuk struktur jaringan heterogen yang terdiri dari atas node dan edge, dalam konteks sistem rekomendasi node mencakup entitas seperti pengguna, film, genre, age, occupation, dan gender. Sementara edge menggambarkan hubungan antar entitas, seperti hasGenre (memiliki genre), hasOccupation (memiliki pekerjaan) atau rates (memberi rating), dan sebuah knowledge graph umumnya direpresentasikan dalam bentuk (h, r, t) yang terdiri dari head, relation, tail, secara sistematis triplet dinyatakan sebagai berikut [16][17]:

$$\mathcal{G} = \{(h, r, t) \mid h \in \mathcal{e}, t \in \mathcal{e}, r \in \mathcal{R}\} \quad (2)$$

Penjelasan:

$\mathcal{G}$ = Sekumpulan triple yang menyimpan informasi hubungan antar entitas

(h, r, t) = kepala, relasi dan ekor dari sebuah triple pengetahuan

$\mathcal{e}$ = Himpunan entitas dalam knowledge graph

$\mathcal{R}$ = Himpunan relasi dalam knowledge graph

Triplet-triplet ini mencerminkan hubungan semantik dan struktural yang akan digunakan sebagai dasar dalam pembuatan edge pada graph.

Pada Penelitian [7] memanfaatkan knowledge graph di sisi pengguna dan item berhasil mencapai AUC 0.982. Dengan memanfaatkan informasi semantik yang kaya dari knowledge graph, model ini dapat mengatasi masalah sparsity data dengan memperkenalkan lebih banyak hubungan semantik yang mendukung akurasi rekomendasi. Selain itu, model ini memperhitungkan atribut pengguna dan bagaimana atribut tersebut mempengaruhi preferensi mereka terhadap item, sehingga menciptakan representasi yang lebih akurat.

2.3.2 Graph Convolutional Network

Graph Convolutional Network (GCN) adalah jenis jaringan saraf yang dirancang untuk bekerja dengan data yang terstruktur dalam bentuk graf, memungkinkan model menangkap informasi dari *node* tetangga melalui mekanisme propagasi informasi [18]. Pada GCN, setiap node direpresentasikan sebagai vektor angka yang disebut embedding. Nilai embedding akan “dipelajari” model selama proses pelatihan agar menjadi representasi yang bermakna [19].

Pada penelitian [4] menerapkan sistem rekomendasi menggunakan GCN. Penelitian berhasil mencapai AUC 0.978 menggunakan dataset Movielens. Secara khusus, pendekatan ini mampu menangkap preferensi unik dari setiap pengguna dengan menghitung skor relevansi hubungan antar *node* menggunakan *dot product* embedding:

$$\pi_r^u = g(u, r) \quad (3)$$

Penjelasan:

π_r^u = Menggambarkan pentingnya hubungan r dengan pengguna u

$g(u, r)$ = Fungsi interaksi antara embedding pengguna dan embedding relasi

Setelah didapat bobot hubungan, skor tersebut akan dinormalisasi menggunakan fungsi *softmax* pada persamaan (4) untuk menjadi bobot probabilitas [4]:

$$\tilde{\pi}_{r,v,e}^u = \frac{\exp(\pi_{r,v,e}^u)}{\sum_{e \in N(v)} \exp(\pi_{r,v,e}^u)} \quad (4)$$

Bobot probabilitas ini yang dijadikan “filter” yang personalisasi saat menghitung representasi lingkungan entitas [4].

2.3.3 Message Passing dan Sum Aggregation

Message passing merupakan penggabungan informasi dimana setiap node menerima "pesan" berupa embedding dari tetangganya, kemudian mengagregasi pesan-pesan tersebut menggunakan fungsi agregasi [20]. Agregasi representasi bertujuan untuk mengumpulkan dan menggabungkan informasi dari *node* tetangga

sehingga model dapat memahami pola dan ketergantungan lokal dalam graf menjadi satu vektor representasi baru [21].

$$\mathbf{agg}_{sum} = \sigma(\mathbf{W} \cdot (\mathbf{v} + \mathbf{v}_{S(v)}^u)) + \mathbf{b} \quad (5)$$

Penjelasan:

$\mathbf{W}$ = Transformasi bobot

$\mathbf{v}$ = Embedding node pusat

$\mathbf{b}$ = Transformasi bias

σ = Nonlinier function

2.3.4 Skor Prediksi

Fungsi prediksi $\mathcal{F}$ memanfaatkan embedding akhir dari pengguna $\mathbf{u}$ dan item $\mathbf{v}$ yang telah diperkaya dengan informasi dari matrik interaksi $\mathbf{Y}$ dan struktur knowledge graph $\mathcal{G}$. Fungsi ini kemudian menghitung probabilitas interaksi antara keduanya [22]:

$$\hat{\mathbf{y}}_{uv} = \mathcal{F}(\mathbf{u}, \mathbf{v} | \boldsymbol{\theta}, \mathbf{Y}, \mathcal{G}) \quad (6)$$

Penjelasan:

$\hat{\mathbf{y}}_{uv}$ = Nilai prediksi probabilitas pengguna $\mathbf{u}$ akan berinteraksi dengan item $\mathbf{v}$

$\mathcal{F}$ = Fungsi untuk memprediksi nilai $\hat{\mathbf{y}}_{uv}$

$\boldsymbol{\theta}$ = Parameter model mengoptimalkan fungsi prediksi

$\mathbf{Y}$ = Matriks interaksi pengguna-item

$\mathcal{G}$ = *Knowledge Graph* memberikan konteks tambahan tentang hubungan antar entitas

2.4 Importance sampling (IS)

Penerapan *Graph Convolutional Network* (GCN) pada graf berskala besar menghadapi tantangan waktu dan biaya memori yang besar karena banyaknya interaksi dan tetangga yang harus diproses, sehingga membuat proses agregasi menjadi sangat lambat. Untuk mengurangi biaya komputasi dan memori maka

penerapan *importance sampling* merupakan arah yang menjanjikan. *Importance Sampling* merupakan teknik pengambilan sampel yang bertujuan untuk memilih *node* atau *edge* yang memiliki bobot lebih besar atau dianggap lebih penting dalam graph. Sampling ini dilakukan terhadap subset *node* atau *edge* yang memiliki informasi signifikan, sehingga proses pelatihan model menjadi lebih efisien dan akurat [5].

$$\hat{\mu}_q = \frac{1}{n} \sum_j \frac{p(\hat{v}_j)}{q(\hat{v}_j)} f(\hat{v}_j) \quad (7)$$

Penjelasan:

n = Jumlah tetangga yang diambil sampelnya

$p(\hat{v}_j)$ = bobot dari *node* atau *edge* $\hat{v}_j$

$f(\hat{v}_j)$ = informasi node $\hat{v}_j$ yang akan dipropagasi

$\hat{v}_j$ = node tetangga yang diambil dari distribusi q

Penelitian [23] penerapan teknik *importance sampling* dilakukan pada GCN. Dalam penelitian ini, diusulkan sebuah metode *layer-wise sampling* dimana dilakukan pengambilan sampel pada sejumlah node untuk setiap lapisan GCN, yang merupakan cara efisien untuk mengurangi jumlah komputasi saat melatih GCN. Diuji pada dataset seperti Cora, Pudmed dan Reddit, penelitian ini berhasil membuktikan bahwa pendekatan ini berhasil menyelesaikan masalah efisiensi dan skalabilitas pada graf yang besar.

2.5 Hyperparameter Tuning

Hyperparameter tuning adalah proses dimana parameter model dilakukan penyesuaian sedemikian rupa untuk meningkatkan kinerja model. Pada KGCN beberapa parameter yang disesuaikan diantaranya:

Tabel II. 1 Hyperparameter Tuning

<i>Embedding Dimension</i>	Ukuran dimensi embedding dalam merepresentasikan isi node
<i>Iteration number</i>	Jumlah perulangan yang dilakukan model
<i>Learning_Rate</i>	Langkah yang diambil untuk memperbarui bobot model selama pelatihan.

Metode hyperparameter yang akan digunakan adalah *GridSearchCV* dimana parameter akan ditentukan secara manual berdasarkan metrik yang sudah dipilih[24][25].

2.6 Metrik Evaluasi

Metrik evaluasi adalah alat untuk mengukur kinerja model pada berbagai tugas seperti klasifikasi biner, multi-kelas, regresi, segmentasi gambar[26]. Berikut metrik evaluasi yang digunakan pada sistem rekomendasi:

a. Recall@K

Merupakan sebuah evaluasi untuk rekomendasi yang diberikan model. Recall@K merepresentasikan berapa banyak item di top_K yang termasuk kedalam *ground_truth_item* [16].

$$Rec@K = \frac{ground_truth_items \cap top_k}{ground_truth_items} \quad (8)$$

b. Precision@K

Menunjukkan jumlah item K di *ground_truth_item* yang termasuk kedalam top_K dan relevan untuk direkomendasikan kepada pengguna [7].

$$Pre@K = \frac{ground_truth_items \cap top_k}{top_k} \quad (9)$$

c. Normalized Discounted Cumulative Gain (NDGC@K)

Digunakan untuk merepresentasikan kualitas peringkat rekomendasi yang dihasilkan, semakin mendekati nilai 1 mengindikasikan kualitas peringkat rekomendasi yang sangat baik [15].

$$NDCG(u) = \frac{DCG@K}{IDCG@K} \quad (10)$$

Dimana didapat dari menghitung *Discounted Cumulative Gain* (DCG) dalam memperhitungkan posisi item, jika nilai DCG tinggi menunjukkan lebih banyak item dengan peringkat tinggi [27] dan *Ideal Discounted Cumulative Gain* (IDCG) yang dijadikan sebagai perbandingan [28].

$$DCG(u) = \sum_{j=1}^n \frac{2^{rel_i} - 1}{\log_2(i + 1)} \quad (11)$$

$$IDCG(u) = \sum_{i=1}^{|rel_k|} \frac{2^{rel_i-1}}{\log_2(i + 1)} \quad (12)$$

Penjelasan:

K = Peringkat maksimum

i = posisi item dalam peringkat

rel_i = nilai relevansi item pada posisi i

d. AUC (Area Under Curve)

Metric yang digunakan untuk mengevaluasi kemampuan model dalam melakukan klasifikasi, nilai yang lebih tinggi menunjukkan kinerja model yang lebih baik [29].

$$AUC = \frac{1}{|R||N|} \sum_{i \in R} \sum_{j \in N} \delta(s_i < s_j) \quad (13)$$

Penjelasan:

R = Himpunan item relevan rating ≥ 4

N = Himpunan item tidak relevan

s_i = Skor prediksi model untuk item i

s_j = Skor prediksi model untuk item j

2.7 Output

Daftar peringkat film yang paling mungkin disukai oleh pengguna. Prosesnya dimulai untuk seorang target pengguna, dimana model yang telah dilatih akan

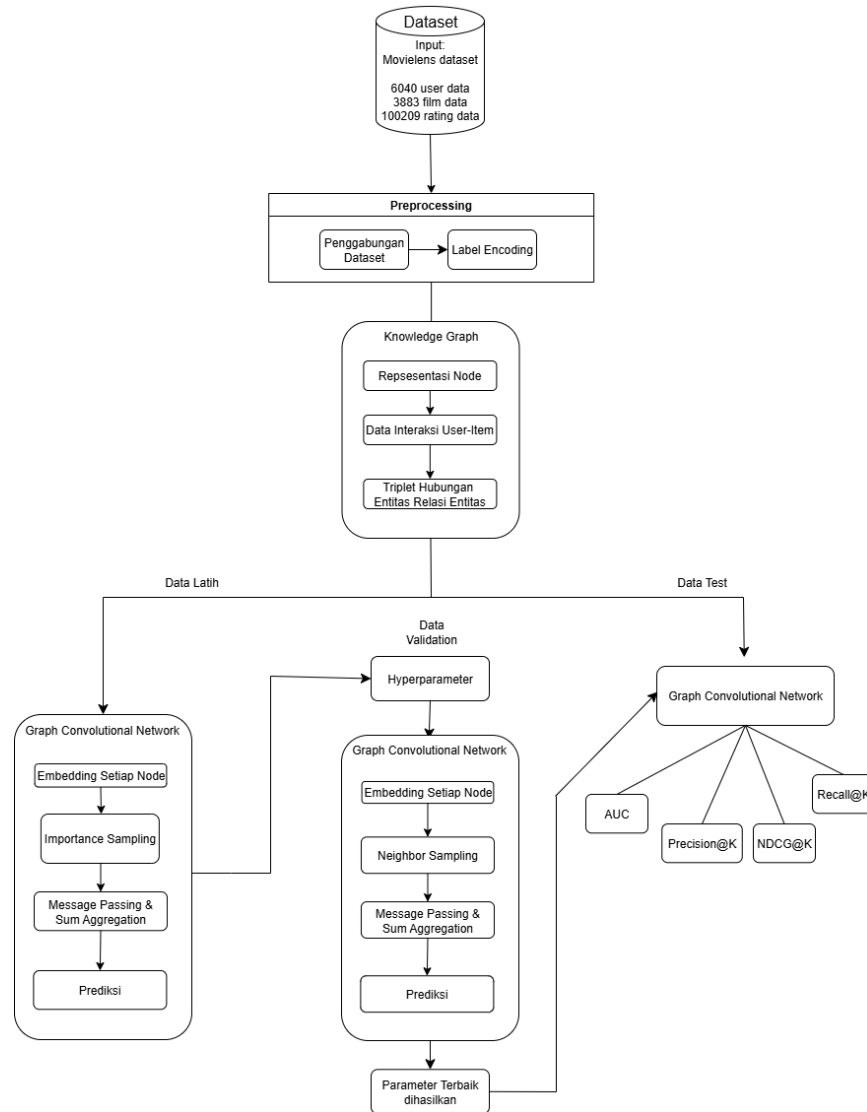
menghitung skor probabilitas interaksi antara pengguna dengan kandidat film yang belum pernah dilihat sebelumnya. Seluruh item kemudian diurutkan berdasarkan skor dari tertinggi ke terendah, dan sistem akan mengambil sejumlah K item teratas untuk ditampilkan [7].

BAB III METODOLOGI PENELITIAN

3.1 Pendekatan Penelitian

Penelitian dilakukan dengan pendekatan kuantitatif, yaitu pendekatan dalam menguji teknik *importance sampling* berbasis bobot edge pada model *Graph Convolutional Network* (GCN). *Importance sampling* dilakukan dengan memilih edge yang memiliki bobot (*edge_weight*) lebih besar dari threshold tertentu, kemudian sampling edge dilakukan secara probabilistik proporsional terhadap bobot tersebut. Teknik ini bertujuan untuk memfokuskan pelatihan pada hubungan yang lebih penting antara pengguna dan item, sehingga meningkatkan efisiensi dan akurasi model. Evaluasi model dilakukan menggunakan metrik evaluasi seperti Precision@K, Recall@K, NDCG@K, dan AUC untuk mengukur kualitas rekomendasi dan mengurangi waktu komputasi selama training. Untuk memberikan menyeluruh, keseluruhan proses divisualisasikan dalam diagram alur pada Gambar III.1

3.2 Metodologi Penelitian



Gambar III. 1 Metode Penelitian

3.3 Dataset

Pada penelitian ini, data bersumber dari dataset publik MovieLens 1M yang terdiri dari 6040 pengguna unik, 3883 film, dan 1.000.209 data rating. Untuk memahami karakteristik pengguna, penelitian ini memanfaatkan data demografis yang rinciannya dapat dilihat pada Tabel III.1.

Tabel III. 1 Atribut Pengguna

No	Atribut	Deskripsi
1	UserID	ID yang mewakili user
2	Gender	Merupakan atribut user yang menjelaskan jenis kelamin user
3	Age	Merupakan atribut user yang berisikan kategori umur user
4	Occupation	Merupakan atribut user yang berisikan kategori pekerjaan user

Selain data pengguna, penelitian ini juga menggunakan data film yang mencakup atribut seperti ID unik, judul, dan genre untuk membangun knowledge graph. Atribut-atribut ini disajikan secara rinci pada Tabel III.2.

Tabel III. 2 Atribut Film

No	Atribut	Deskripsi
1	MovieID	ID yang mewakili film tertentu
2	Title	Merupakan Atribut judul film
3	Genres	Merupakan atribut film yang mengkategorikan genre film

Kemudian digunakan juga data rating yang menggambarkan preferensi pengguna terhadap film, interaksi digambarkan dengan rating yang diberikan oleh pengguna terhadap film yang telah ditonton. Struktur dari data rating disajikan pada Tabel III.3.

Tabel III. 3 Data Rating

No	Atribut	Deskripsi
1	UserID	ID yang mewakili user tertentu
2	MovieID	ID yang mewakili film tertentu
3	Ratings	Penilaian yang diberikan user terhadap film (1-5 bintang)

3.4 Preprocessing Data

Preprocessing data merupakan tahap untuk membersihkan data agar data tidak mengurangi akurasi model sehingga data mentah bisa dipakai saat dianalisis, dilakukan beberapa tahap preprocessing:

3.4.1 Penggabungan Dataset

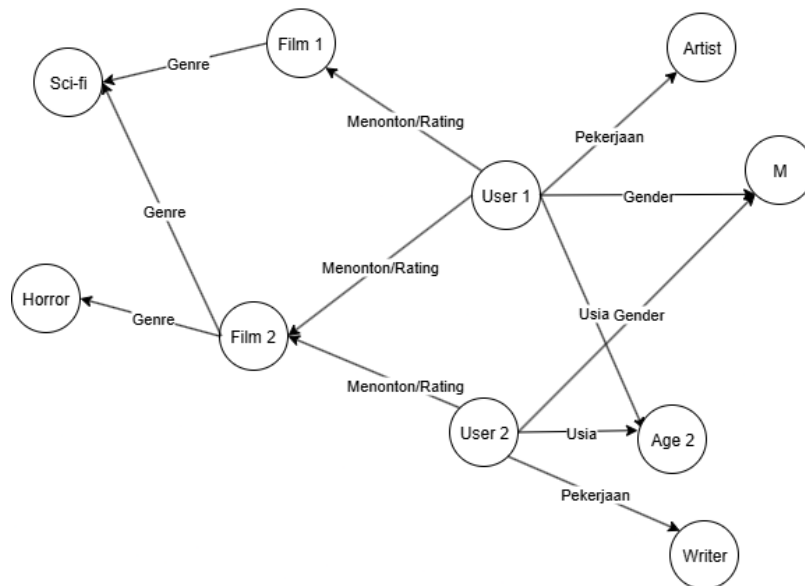
Pada tahap ini, beberapa dataset digabungkan menjadi satu kesatuan berdasarkan kesamaan MovieID dan UserID. Proses ini bertujuan untuk memperoleh informasi yang lebih lengkap mengenai interaksi pengguna terhadap film, seperti rating yang diberikan serta genre film yang disukai. Dengan melakukan penggabungan ini, analisis dapat mengungkap preferensi pengguna berdasarkan atribut-atribut tertentu (seperti gender, usia, atau pekerjaan), sehingga dapat diketahui pola kecenderungan pengguna dengan karakteristik tertentu terhadap genre film tertentu. Hasil penggabungan ini menjadi fondasi penting dalam membangun sistem rekomendasi yang personal dan relevan.

3.4.2 Label Encoding

Label encoding untuk mengubah setiap data kategorik menjadi data numerik unik yang memiliki makna tanpa memperhatikan urutan atau tingkatan [30][31], misalnya pada kolom gender terdapat 2 kategori yaitu, M dan F, melalui label encoding, kategori tersebut akan di-*transform* menjadi 0 dan 1.

3.5 Implementasi Knowledge Graph

Setelah melalui tahap preprocessing, langkah selanjutnya adalah membangun Knowledge Graph (KG) yang digunakan untuk merepresentasikan hubungan kompleks antara user dan item, serta atribut-atribut tambahan dari masing-masing entitas seperti pada Gambar III.2.



Gambar III. 2 Arsitektur Knowledge Graph

3.5.1 Representasi Node

Langkah awal dalam membangun KG adalah membuat representasi node untuk setiap entitas yang terlibat. Entitas ini berasal dari atribut-atribut user maupun film, antara lain:

- UserID
- MovieID
- Genre
- Age
- Occupation
- Gender

Setiap entitas akan di-*mapping* ke dalam ID numerik unik untuk menghindari duplikasi dan memastikan konsistensi saat membangun graph. Representasi node

ini bertujuan untuk menciptakan struktur graph heterogen di mana setiap jenis entitas dapat dikenali secara eksplisit.

3.5.2 Data Interaksi User-Item

Selanjutnya, dibentuk data interaksi eksplisit antara pengguna dan film yang direpresentasikan menggunakan persamaan (1). Dimana matriks interaksi ini $Y \in \mathbb{R}^{6040 \times 3883}$, dimana nilai Y berisikan nilai rating dari 1 hingga 5. Nilai rating ini akan digunakan sebagai bobot edge yang menghubungkan antara *node* pengguna dan film.

3.5.3 Triplet Hubungan Entitas Relasi Entitas

Terakhir, semua hubungan antar entitas didalam KG akan dibentuk dalam format triplet (*head, relation, tail*) menggunakan persamaan (2). Triplet-triplet ini mencerminkan hubungan yang menjadi dasar dalam pembuatan edge pada graf, seperti yang digambarkan pada Tabel III.4. Sebagai contoh triplet:

Tabel III. 4 Triplet Hubungan Entitas Relasi Entitas

Head (h)	Relation (r)	Tail (t)
U1	<i>rates</i>	M1
U1	<i>rates</i>	M2
U1	<i>has_gender</i>	Male
U1	<i>has_age</i>	2
U1	<i>has_occupation</i>	Programmer
M1	<i>has_genre</i>	Sci-Fi
M1	<i>has_genre</i>	Action

3.6 Splitting Data

Dataset dibagi menjadi tiga bagian, yaitu *train*, *validation* dan *test* dengan proporsi 80:10:10. Pada data *train* digunakan untuk melatih model, data *validation* digunakan untuk melatih model melalui hyperparameter yang sudah ditetapkan, dan data *test* digunakan untuk mengevaluasi model.

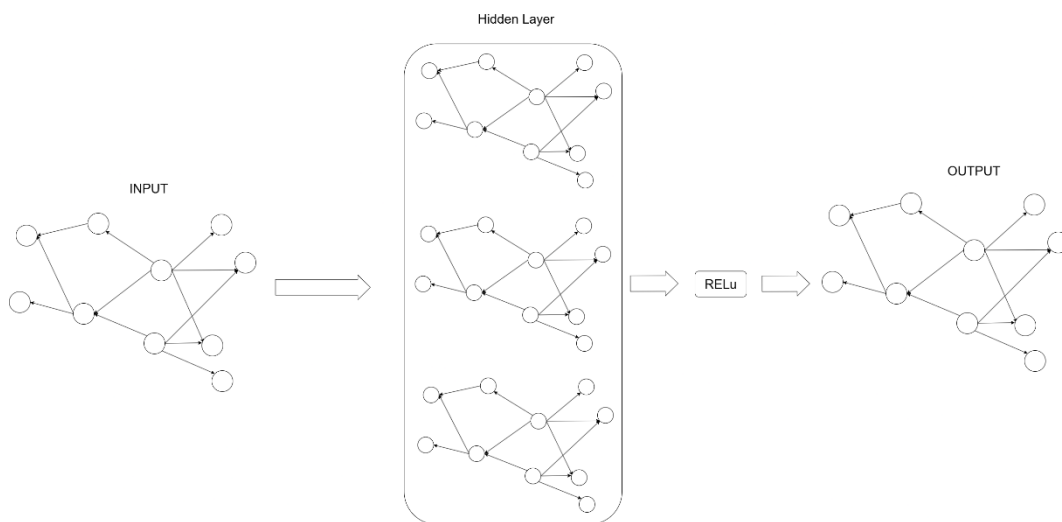
3.7 Implementasi Graph Convolutional Network

Setelah dilakukan *splitting data*, model *Graph Convolutional Network* (GCN) digunakan untuk mempelajari representasi *embedding* dari struktur graf yang sudah

terbentuk. GCN bekerja dengan menyebarkan informasi antar *node* melalui beberapa lapisan, sehingga memungkinkan model menangkap hubungan kompleks antara pengguna, film dan atribut-atributnya. Proses penyebaran informasi secara berlapis ini menjadi mekanisme GCN, dimana setiap lapisan akan memperbarui representasi *node* dengan menggabungkan informasi dari tetangga.

Untuk menemukan kombinasi hyperparameter yang optimal secara efisien, penelitian ini menerapkan mekanisme Early Stopping selama proses hyperparameter tuning. Pelatihan untuk setiap kombinasi parameter akan dihentikan secara otomatis jika metrik evaluasi AUC pada data validasi tidak menunjukkan peningkatan setelah periode epoch tertentu (*patience*). Pendekatan ini memastikan bahwa hanya hyperparameter yang menjanjikan yang dievaluasi secara penuh, sehingga dapat menghemat waktu komputasi secara signifikan.

Berikut arsitektur dari GCN pada Gambar III.3.



Gambar III. 3 Arsitektur Graph Convolutional Network

Tahapan yang dilakukan GCN dalam penelitian sebagai berikut:

3.7.1 Embedding Awal

Pada tahap awal proses pelatihan, setiap node dalam graph, seperti pengguna, *gender*, *age*, *occupation*, film, genre, diisi vektor embedding dengan dimensi 64 dan nilainya diberikan secara acak. Pemberian embedding awal secara acak ini bertujuan sebagai titik awal bagi model untuk mempelajari informasi yang bermakna. Untuk memberikan gambaran yang lebih jelas, Untuk mengilustrasikan

konsep ini, Tabel III.5. berikut menyajikan contoh embedding awal dengan 4 dimensi.

Tabel III. 5 Embedding Awal

Node	Embedding
U1	[0.1, 0.8, 0.2, 0.5]
M1	[0.9, 0.2, 0.1, 0.3]
M2	[0.8, 0.2, 0.2, 0.2]
Programmer	[0.2, 0.7, 0.6, 0.3]
2	[0.5, 0.5, 0.4, 0.4]
Male	[0.6, 0.3, 0.3, 0.6]
Sci-Fi	[0.9, 0.9, 0.2, 0.2]
Action	[0.8, 0.2, 0.1, 0.1]

3.7.2 Importance Sampling

Setelah pemberian *embedding* awal, selanjutnya diterapkan *importance sampling* untuk membantu dalam mengatasi tantangan efisiensi pada graf berskala besar. Berbeda dengan *uniform sampling* yang memilih tetangga secara acak, *importance sampling* bertujuan untuk memilih tetangga yang paling relevan dan informatif. Proses ini melalui beberapa langkah:

1. Perhitungan skor relevansi menggunakan persamaan (3) antara *embedding* pengguna dan *embedding* relasi, kemudian dikalikan dengan bobot rating.
2. Skor relevansi kemudian diubah menjadi nilai probabilitas menggunakan fungsi softmax pada persamaan (4)
3. Dari distribusi probabilitas yang didapat, dipilih top-5 tetangga dengan skor tertinggi untuk setiap jenis relasi.

Untuk memberikan gambaran yang jelas mengenai implementasi dari langkah diatas, berikut contoh perhitungan menggunakan data dari Tabel III.5.

1. Menghitung skor relevansi

$$\begin{aligned}
 score_{U1, M1} &= e(U1) * e(M1) \\
 &= (0.1)(0.9) + (0.8)(0.2) + (0.2)(0.1) + (0.5)(0.3) \\
 &= 0.09 + 0.16 + 0.02 + 0.15 = 0.42 \times 5 = 2.10
 \end{aligned}$$

$$\begin{aligned}
score_{U1, M2} &= e(U1) * e(M2) \\
&= (0.1)(0.8) + (0.8)(0.2) + (0.2)(0.2) + (0.5)(0.2) \\
&= 0.08 + 0.16 + 0.04 + 0.10 = 0.38 \times 4 = 1.52 \\
score_{U1, Programmer} &= e(U1) * e(Programmer) \\
&= (0.1)(0.2) + (0.8)(0.7) + (0.2)(0.6) + (0.5)(0.3) \\
&= 0.02 + 0.56 + 0.12 + 0.15 = 0.85 \\
score_{U1, 2} &= e(U1) * e(2) \\
&= (0.1)(0.5) + (0.8)(0.5) + (0.2)(0.4) + (0.5)(0.4) \\
&= 0.05 + 0.4 + 0.08 + 0.20 = 0.73 \\
score_{U1, Male} &= e(U1) * e(Male) \\
&= (0.1)(0.6) + (0.8)(0.3) + (0.2)(0.3) + (0.5)(0.6) \\
&= 0.06 + 0.24 + 0.06 + 0.30 = 0.66
\end{aligned}$$

2. Hitung probabilitas (Softmax)

$$e^{2.10} \approx 8.17$$

$$e^{1.52} \approx 4.57$$

$$e^{0.85} \approx 2.34$$

$$e^{0.73} \approx 2.08$$

$$e^{0.66} \approx 1.94$$

$$Total = 19.10$$

Sehingga didapatkan skor probabilitas:

$$softmax(U1, M1) = \frac{8.17}{19.10} \approx 0.43$$

$$softmax(U1, M2) = \frac{4.57}{19.10} = 0.24$$

$$softmax(U1, Programmer) = \frac{2.34}{19.10} = 0.12$$

$$softmax(U1, 2) = \frac{2.08}{19.10} = 0.11$$

$$softmax(U1, Male) = \frac{1.94}{19.10} = 0.10$$

Berdasarkan distribusi probabilitas yang dihasilkan, *node-node* tetangga dengan bobot tertinggi dipilih sebagai sampel representatif untuk proses *message*

passing pada lapisan pertama. Dilanjut ke *layer* ke-2 untuk mengupdate bobot tetangga M1:

Tetangga M1:

- Sci-Fi
- Action

Maka

$$\begin{aligned}
 scoreM1, Sci - Fi &= e(M1) \times e(Sci - Fi) \\
 &= (0.9)(0.9) + (0.2)(0.9) + (0.1)(0.2) + (0.3)(0.2) \\
 &= 0.81 + 0.18 + 0.2 + 0.6 = 1.07
 \end{aligned}$$

$$\begin{aligned}
 scoreM1, Action &= e(M1) * e(Action) \\
 &= (0.9)(0.8) + (0.2)(0.2) + (0.1)(0.1) + (0.3)(0.1) \\
 &= 0.72 + 0.04 + 0.01 + 0.03 = 0.80
 \end{aligned}$$

Softmax *layer* ke-2

$$e^{1.07} \approx 2.92$$

$$e^{0.80} \approx 2.23$$

$$Total = 5.15$$

Probabilitas untuk tetangga di *layer* k-2:

$$softmax(M1, Sci - Fi) = \frac{2.92}{5.15} \approx 0.57$$

$$softmax(M1, Action) = \frac{2.23}{5.15} \approx 0.43$$

3.7.3 Message Passing dan Sum Aggregation

Setelah bobot relevansi setiap tetangga dihitung, GCN akan melakukan proses *message passing*. Dalam proses ini, informasi dari *node* tetangga yang telah terpilih melalui *importance sampling* akan dipropagasikan untuk memperbarui *embedding* dari *node* pusat. Penelitian ini menggunakan *sum aggregation*, di mana nilai *embedding* dari tetangga yang relevan akan dijumlahkan untuk menghasilkan *embedding* agregat, kemudian digabungkan dengan *embedding* asli dari *node* pusat untuk menghasilkan *embedding* baru melalui persamaan (5). Berikut contoh

perhitungan *message passing* untuk memperbarui embedding U1 menggunakan bobot probabilitas yang telah dihitung sebelumnya:

$$v_{M1,new} = 0.57 * [0.9, 0.9, 0.2, 0.2] + 0.43 * [0.8, 0.2, 0.1, 0.1]$$

$$v_{M1,new} = [0.513, 0.513, 0.114, 0.114] + [0.344, 0.086, 0.043, 0.043]$$

$$v_{M1,new} = [0.857, 0.599, 0.157, 0.157]$$

Aggregasi *layer* ke-1 ke U1:

$$v_{U1,new} = 0.43 \times v_{M1,new} + 0.24 \times v_{M2} + 0.12 \times v_{programmer} + 0.11 \times v_2 \\ + 0.10 \times v_{Male}$$

$$v_{U1,new} = 0.43 \times [0.857, 0.599, 0.157, 0.157] + 0.24 \times [0.8, 0.2, 0.2, 0.2] \\ + 0.12 \times [0.2, 0.7, 0.6, 0.3] + 0.11 \times [0.5, 0.5, 0.4, 0.4] \\ + 0.10 \times [0.6, 0.3, 0.3, 0.6]$$

$$v_{U1,new} = [0.368, 0.257, 0.067, 0.067] + [0.192, 0.048, 0.048, 0.048] \\ + [0.024, 0.084, 0.072, 0.036] + [0.055, 0.055, 0.044, 0.044] \\ + [0.06, 0.03, 0.03, 0.06]$$

Hasil akhir embedding U1 setelah aggregasi [5,5]:

$$v_{U1,new} = [0.700, 0.474, 0.262, 0.256]$$

3.7.4 Prediksi

Setelah proses *message passing* selesai dan semua *embedding* pengguna dan film telah diperbarui menjadi representasi baru, model akan menghitung skor prediksi akhir. Skor ini merepresentasikan probabilitas interaksi antara seorang pengguna u dan sebuah v . Perhitungan ini dilakukan menggunakan persamaan (6). Berikut perhitungan skor prediksi interaksi antara U1 dan M1 menggunakan *embedding* baru yang telah diperbarui dari proses *message passing* sebelumnya:

$$\text{score}(U1, M1) \\ = (0.700 * 0.856) + (0.47 * 0.599) + (0.26 * 0.157) + (0.26 * 0.157) \\ = 0.599 + 0.281 + 0.040 + 0.041$$

$$= 0.921$$

Berdasarkan hasil perhitungan skor prediksi interaksi antara $U1$ dan $M1$, nilai yang diperoleh sebesar 0.921 menunjukkan bahwa film $M1$ memiliki tingkat preferensi yang tinggi untuk user $U1$ dibandingkan dengan $M2$, sehingga film $M1$ direkomendasikan lebih tinggi kepada user $U1$.

3.8 Perhitungan Metrik Evaluasi

Daftar rekomendasi dari model: [M1, M2, M3, M4, M5, M6, M7, M8, M9, M10]

Daftar film relevan sebenarnya: [M1, M3, M5, M8, M10]

3.8.1 Recall@10

Dilakukan perhitungan apakah model dapat menemukan seluruh film yang disukai oleh pengguna dan memasukkannya kedalam daftar rekomendasi atau tidak.

$$Recall@10 = \frac{5}{5} = 1$$

Model berhasil menemukan 100% film yang disukai oleh pengguna dan dimasukkan kedalam daftar rekomendasi

3.8.2 Precision@10

Kemudian dilakukan perhitungan berapa banyak model menemukan film yang masuk kedalam top-10 rekomendasi dan relevan untuk pengguna

$$Precision@10 = \frac{5}{10} = 0.5$$

Dari daftar rekomendasi, model hanya berhasil menemukan 50% film yang benar-benar relevan untuk user

3.8.3 NDCG@K

Dilakukan perhitungan DCG@10 menggunakan persamaan (11), dimana relevan = 1 jika relevan dan 0 jika tidak relevan.

Tabel III. 6 Evaluasi DCG@K

Posisi (i)	Rekomendasi	Relevansi	$\frac{relevansi}{\log_2(i+1)}$	Hasil
1	M1	1	$\frac{1}{\log_2(1+1)}$	1.000

2	M2	0	$\frac{0}{\log_2(2+1)}$	0
3	M3	1	$\frac{1}{\log_2(3+1)}$	0.500
4	M4	0	$\frac{0}{\log_2(4+1)}$	0
5	M5	1	$\frac{1}{\log_2(5+1)}$	0.387
6	M6	0	$\frac{0}{\log_2(6+1)}$	0
7	M7	0	$\frac{0}{\log_2(7+1)}$	0
8	M8	1	$\frac{1}{\log_2(8+1)}$	0.315
9	M9	0	0	0
10	M10	1	$\frac{1}{\log_2(10+1)}$	0.289
Total				2.491

Kemudian gunakan IDCG pada persamaan (12) untuk menghitung urutan rekomendasi.

Tabel III. 7 Evaluasi IDCG@K

Posisi (i)	Rekomendasi	Relevansi	$\frac{relevansi}{\log_2(i+1)}$	Hasil
1	M1 (Relevan)	1	$\frac{1}{\log_2(1+1)}$	1.000
2	M3 (Relevan)	1	$\frac{1}{\log_2(2+1)}$	0.631
3	M5 (Relevan)	1	$\frac{1}{\log_2(3+1)}$	0.500
4	M8 (Relevan)	1	$\frac{1}{\log_2(4+1)}$	0.431

5	M10 (Relevan)	1	$\frac{1}{\log_2(5 + 1)}$	0.387
Total				2.949

Kemudian gunakan persamaan (10) untuk mendapatkan nilai $NDCG@10$, maka:

$$NDCG@10 = \frac{2.491}{2.949} = 0.845$$

Maka kualitas urutan rekomendasi yang diberikan model mencapai 84,5%.

3.8.4 AUC

AUC bekerja dengan membandingkan skor dari setiap film relevan dengan film yang tidak relevan, dimana dibuat kemungkinan pasangannya. Misalkan model telah memberikan skor prediksi untuk 10 film dan dari data sebelumnya diketahui bahwa terdapat 5 film yang relevan (Positif) dan 5 lainnya tidak relevan (Negatif) seperti pada Tabel III.8.

Tabel III. 8 Evaluasi AUC

Film	Skor	Label
M1	0.95	Positif
M2	0.60	Negatif
M3	0.90	Positif
M4	0.65	Negatif
M5	0.80	Positif
M6	0.50	Negatif
M7	0.40	Negatif
M8	0.70	Positif
M9	0.55	Negatif
M10	0.52	Positif

1. Hitung Total Pasangan

AUC bekerja dengan membandingkan skor dari setiap film positif dengan film negatif. Dalam skenario ini berarti total pasangan yang akan dibandingkan sebanyak:

$$\text{Total Pasangan} = 5 (\text{Positif}) \times 5 (\text{Negatif}) = 25$$

2. Hitung Jumlah Pasangan yang diurutkan benar

- M1 (0.95)

M2 (0.60)

M4 (0.65)

M6 (0.50)

M7 (0.40)

M9 (0.55)

Film M1 mengungguli skor film yang tidak relevan

- M3 (0.90)

M2 (0.60)

M4 (0.65)

M6 (0.50)

M7 (0.40)

M9 (0.55)

Film M3 mengungguli skor film yang tidak relevan

- M5 (0.80)

M2 (0.60)

M4 (0.65)

M6 (0.50)

M7 (0.40)

M9 (0.55)

Film M5 mengungguli skor film yang tidak relevan

- M8 (0.70)

M2 (0.60)

M4 (0.65)

M6 (0.50)

M7 (0.40)

M9 (0.55)

Film M8 mengguguli semua skor film yang tidak relevan

- M10 (0.52)

M8 (0.70)

M2 (0.60)

M4 (0.65)

M6 (0.50)

M7 (0.40)

M9 (0.55)

Film M10 hanya mengguguli film M7 dan M6 saja, Maka:

$$5(M1) + 5(M3) + 5(M5) + 5(M8) + 2(M10) = 22$$

3. Hitung Skor AUC

$$AUC = \frac{22}{25} = 0.88$$

Model dapat membedakan dengan baik antara film yang relevan dan tidak relevan untuk pengguna yang ditunjukkan dengan pemberian skor lebih tinggi pada film yang disukai pengguna dibandingkan film yang tidak disukai.

BAB IV HASIL DAN PEMBAHASAN

4.1 Hasil Preprocessing Data dan Pembentukan Graf

Berdasarkan tahapan implementasi yang telah dirancang pada BAB III Metodologi Penelitian, bab ini akan menyajikan dan membahas hasil dari setiap proses yang telah dilakukan. Pembahasan akan dimulai dari penyajian karakteristik dataset awal, diikuti oleh hasil dari setiap langkah pra-pemrosesan, pembentukan Knowledge Graph, hingga hasil akhir dari pelatihan dan pengujian model.

4.1.1 Dataset

Analisis dimulai dari dataset mentah yang diperoleh dari platform MovieLens. Dataset ini terbagi menjadi tiga file utama. Data pertama yang digunakan adalah data film, yang mencakup atribut seperti ID unik, judul, dan genre, dimana dataset movies ditemukan 3883 data film. Tampilan dari data film ini disajikan pada Gambar IV.1 berikut.

MovieID		Title	Genres
0	1	Toy Story (1995)	Animation Children's Comedy
1	2	Jumanji (1995)	Adventure Children's Fantasy
2	3	Grumpier Old Men (1995)	Comedy Romance
3	4	Waiting to Exhale (1995)	Comedy Drama
4	5	Father of the Bride Part II (1995)	Comedy
...	...	...	...
3878	3948	Meet the Parents (2000)	Comedy
3879	3949	Requiem for a Dream (2000)	Drama
3880	3950	Tigerland (2000)	Drama
3881	3951	Two Family House (2000)	Drama
3882	3952	Contender, The (2000)	Drama Thriller
3883 rows x 3 columns			

Gambar IV. 1 Data Film

Selanjutnya, terdapat data rating yang menggambarkan interaksi antara pengguna dan film. Dataset ini mencakup total 1.000.209 data rating yang diberikan oleh pengguna, dengan skala penilaian dari 1 hingga 5. Rincian dari struktur data rating disajikan pada Gambar IV.2.

	UserID	MovieID	Rating
0	1	1193	5
1	1	661	3
2	1	914	3
3	1	3408	4
4	1	2355	5
...	...	...	...
1000204	6040	1091	1
1000205	6040	1094	5
1000206	6040	562	5
1000207	6040	1096	4
1000208	6040	1097	4
1000209 rows x 3 columns			

Gambar IV. 2 Data Rating

Untuk menciptakan rekomendasi yang bersifat personal, penelitian ini memanfaatkan data demograsi dari pengguna, dimana ditemukan 6040 data dengan 4 atribut yaitu, *UserID*, *Gender*, *Age*, *Occupation*. Model akan memanfaatkan data tersebut untuk memahami preferensi dari masing-masing pengguna. Tampilan dari data pengguna dapat dilihat pada Gambar IV.3.

	UserID	Gender	Age	Occupation
0	1	F	1	10
1	2	M	56	16
2	3	M	25	15
3	4	M	45	7
4	5	M	25	20
...	...	...	...	...
6035	6036	F	25	15
6036	6037	F	45	1
6037	6038	F	56	1
6038	6039	F	45	0
6039	6040	M	25	6
6040 rows x 4 columns				

Gambar IV. 3 Data Pengguna

4.1.2 Penggabungan Dataset

Setelah mengetahui isi dari setiap dataset, langkah selanjutnya adalah melakukan penggabungan dataset menjadi satu dataset yang utuh. Proses penggabungan dilakukan berdasarkan kesamaan *UserID* dan *MovieID* untuk

mengetahui informasi demografis pengguna, detail film, dan data rating. Hasil dari penggabungan dataset menghasilkan total 1.000.208 data interaksi dengan 8 atribut, seperti pada Gambar IV.4 berikut.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000209 entries, 0 to 1000208
Data columns (total 8 columns):
#   Column      Non-Null Count  Dtype
---  -
0   UserID      1000209 non-null  int64
1   Gender      1000209 non-null  object
2   Age         1000209 non-null  int64
3   Occupation  1000209 non-null  int64
4   MovieID     1000209 non-null  int64
5   Rating      1000209 non-null  int64
6   Title       1000209 non-null  object
7   Genres      1000209 non-null  object
```

Gambar IV. 4 Penggabungan Dataset

4.1.3 Label Encoding

Kemudian dilakukan label encoding pada kolom gender, dengan merubah atribut kategorik menjadi bentuk numerik. Pada atribut gender female = 0 dan male = 1, perubahan ini dilakukan berdasarkan urutan abjad, dimana huruf 'F' pada kata Female muncul lebih awal dalam alfabet dibandingkan huruf 'M' pada kata Male. Hasil dari label encoding disajikan pada Gambar IV.5 berikut.

	UserID	Gender	Age	Occupation	MovieID	Rating	Title	Genres
0	1	0	1	10	1193	5	One Flew Over the Cuckoo's Nest (1975)	Drama
1	1	0	1	10	661	3	James and the Giant Peach (1996)	Animation Children's Musical
2	1	0	1	10	914	3	My Fair Lady (1964)	Musical Romance
3	1	0	1	10	3408	4	Erin Brockovich (2000)	Drama
4	1	0	1	10	2355	5	Bug's Life, A (1998)	Animation Children's Comedy
...	...	...	...	...	...	...	...	...
1000204	6040	1	25	6	1091	1	Weekend at Bernie's (1989)	Comedy
1000205	6040	1	25	6	1094	5	Crying Game, The (1992)	Drama Romance War
1000206	6040	1	25	6	562	5	Welcome to the Dollhouse (1995)	Comedy Drama
1000207	6040	1	25	6	1096	4	Sophie's Choice (1982)	Drama
1000208	6040	1	25	6	1097	4	E.T. the Extra-Terrestrial (1982)	Children's Drama Fantasy Sci-Fi

Gambar IV. 5 Label Encoding

4.1.4 Inisialisasi Knowledge Graph

4.1.4.1 Representasi Node

Setelah penerapan label encoding, langkah pertama adalah membentuk representasi node dari semua entitas dalam dataset. Entitas tersebut dikelompokkan menjadi beberapa jenis:

- Node User: merepresentasikan ID pengguna
- Node Movie: merepresentasikan ID film
- Node Genre: merepresentasikan genre seperti Action, Drama, Comedy
- Node Gender: merepresentasikan laki-laki atau Perempuan
- Node Age: merepresentasikan kelompok umur
- Node Occupation: merepresentasikan pekerjaan

Setiap entitas di-*mapping* ke ID numerik unik untuk mencegah terjadinya duplikasi data, dan membentuk struktur awal graph heterogen.

4.1.4.2 Data Interaksi User-Item

Dari data rating, dibentuk matriks interaksi menggunakan persamaan (1), sehingga didapatkan $Y \in \mathbb{R}^{6040 \times 3883}$. Elemen Y berisi nilai rating 1-5 yang akan digunakan untuk memberikan bobot pada edge antara node pengguna dan node film pada graph.

4.1.4.3 Triplet Hubungan Entitas Relasi Entitas

Selanjutnya, dibentuk relasi antar node di dalam graf menggunakan persamaan (2), sehingga menghasilkan hubungan antara entitas. Triplet hubungan ini digunakan sebagai *Knowledge Graph* untuk menyimpan relasi antar entitas yang merepresentasikan informasi tambahan selain user-item. Dalam proses implementasi, beberapa tipe relasi utama yang berhasil dibentuk pada *Knowledge Graph* disajikan pada Tabel IV.1 berikut.

Tabel IV. 1 Triplet Hubungan

<i>Head</i>	<i>Relation</i>	<i>Tail</i>
<i>user</i>	<i>rates</i>	<i>movie</i>
<i>user</i>	<i>has_gender</i>	<i>gender</i>
<i>user</i>	<i>has_age</i>	<i>age</i>
<i>user</i>	<i>has_occupation</i>	<i>occupation</i>
<i>movie</i>	<i>has_genre</i>	<i>genre</i>

4.1.5 Splitting Dataset

Dataset dibagi menjadi tiga bagian, yaitu *train*, *validation* dan *test* dengan proporsi 80:10:10. Pada data *train* digunakan untuk melatih model, data *validation* digunakan untuk melatih model melalui hyperparameter yang sudah ditetapkan, dan data *test* digunakan untuk mengevaluasi model.

4.2 Inisialisasi Model Graph Convolutional Network

Sebelum proses pelatihan dimulai, setiap node di dalam KG diberikan sebuah representasi nilai *embedding* acak berdimensi 64. *Embedding* ini berfungsi sebagai titik awal model untuk memperbarui nilai *embedding* agar dapat menangkap informasi yang lebih bermakna selama proses pelatihan.

4.3 Pelatihan Model

Model awal dilatih terlebih dahulu menggunakan *data training* menggunakan parameter pada tabel berikut.

Tabel IV. 2 Parameter Awal Model

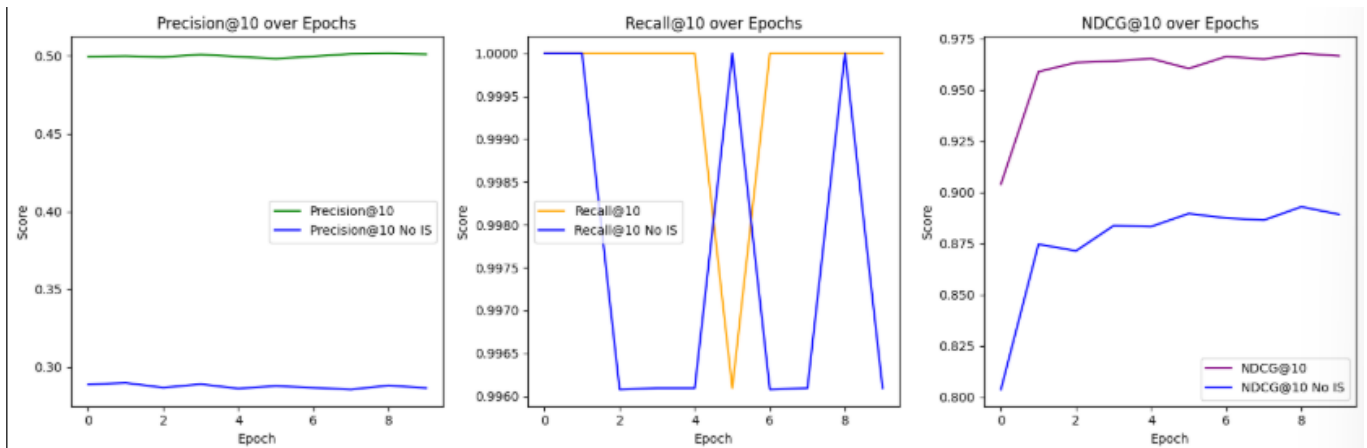
Parameter	Nilai
Optimizer	Adam
Batch Size	128
Learning Rate	0.005
Weight Decay	0.0001
Embedding Size	64
Model Layers	2

Model dilatih selama 10 epoch dengan menghasilkan sebanyak $k = 10$ item rekomendasi pada setiap iterasi pelatihan. Hasil penelitian menunjukkan bahwa penerapan teknik importance sampling mampu meningkatkan performa model secara signifikan dibandingkan dengan uniform sampling, khususnya pada metrik evaluasi Precision@K, Recall@K, NDCG@K, dan AUC. Hasil lebih lengkap terdapat pada Tabel IV.3 dibawah ini.

Tabel IV. 3 Perbandingan Waktu Komputasi Training

Epoch	Importance Sampling		Uniform Sampling	
	Waktu (s)	AUC	Waktu (s)	AUC
1	219.55s	0.6843	199.98s	0.6465
2	223.82s	0.8393	197.19s	0.7766
3	441.95s	0.8485	478.49s	0.7866
4	536.63s	0.8521	513.62s	0.7926
5	617.53s	0.8554	592.25s	0.7953
6	663.63s	0.8586	626.83s	0.7969
7	699.07s	0.8616	664.86s	0.7986
8	735.14s	0.8620	690.11s	0.7988
9	761.97s	0.8628	694.50s	0.7994
10	768.45s	0.8636	722.15s	0.8004

Penerapan importance sampling terbukti memberikan nilai AUC yang lebih tinggi dibandingkan uniform sampling, yang menunjukkan kemampuan model dalam membedakan item relevan dan tidak relevan secara lebih efektif. Meskipun demikian, teknik ini memerlukan waktu komputasi yang lebih lama pada setiap epoch, menandakan adanya *trade-off* antara akurasi dan efisiensi waktu dalam proses pelatihan. Hal ini disebabkan oleh adanya proses tambahan untuk menghitung distribusi bobot penting dari edges sebelum dilakukan proses sampling. Meskipun memerlukan waktu komputasi yang lebih tinggi, peningkatan performa evaluasi yang signifikan menjadikan keuntungan, karena akurasi dan relevansi hasil rekomendasi yang dihasilkan oleh model menjadi lebih optimal. Berikut perbandingan metrik Precision@K, Recall@K, NDCG@K selama 10 epoch pada gambar berikut.



Gambar IV. 6 Metrik Evaluasi

Hasil menunjukkan perbandingan kinerja model dengan menerapkan *importance sampling* dan *uniform sampling* selama pelatihan. Pada metrik Precision@10, model dengan *importance sampling* secara konsisten memperoleh nilai 0.50 lebih tinggi dibandingkan model yang menerapkan *uniform sampling* yang hanya mencapai 0.28. Hasil ini mengindikasikan bahwa *importance sampling* secara signifikan meningkatkan kemampuan model dalam merekomendasikan item yang relevan pada 10 posisi teratas. Pada metrik Recall@10, model yang menerapkan *importance sampling* memiliki performa yang cukup stabil sebesar 1.000 walaupun sempat drop ke 0.9963, sedangkan pada model yang menerapkan *uniform sampling* mendapatkan nilai yang sedikit lebih rendah sebesar 0.9963. Hal ini menunjukkan bahwa *importance sampling* membantu model dalam menemukan hampir seluruh film relevan dalam top rekomendasi.

Pada metrik NDCG@10 menunjukkan perbedaan yang cukup signifikan, model yang menerapkan *importance sampling* mampu mendapatkan skor 0.96, sedangkan model yang menerapkan *uniform sampling* hanya mendapatkan skor 0.88. Ini menunjukkan bahwa model yang menerapkan *importance sampling* dapat menempatkan posisi item yang sangat relevan di posisi tertinggi dalam daftar rekomendasi.

4.4 Hyperparameter

Selanjutnya, dilakukan proses hyperparameter tuning untuk menemukan kombinasi parameter terbaik. Seperti yang telah dijelaskan pada Bab III, proses ini

memanfaatkan mekanisme early stopping untuk setiap kombinasi parameter yang diuji. Hal ini bertujuan agar setiap set parameter dievaluasi secara efisien pada titik performa puncaknya pada data validasi, sehingga penentuan hyperparameter terbaik menjadi lebih valid. Parameter yang diuji disajikan pada tabel berikut.

Tabel IV. 4 Hyperparameter Model

Parameter	Nilai
Learning_Rate	0.001, 0.01, 0.05
Embedding Size	32
Epoch	20

Berdasarkan hasil percobaan hyperparameter dengan optimizer Adam, kombinasi parameter terbaik diperoleh pada learning rate sebesar 0.001. Pada data validasi untuk *importance sampling*, konfigurasi ini menghasilkan Precision@10 sebesar 0.2904, Recall@10 sebesar 1.0000, NDCG@10 sebesar 0.8513, dan AUC sebesar 0.7459. Sementara itu, pada learning rate 0.01 hanya diperoleh AUC sebesar 0.7167, dan pada learning rate 0.05 AUC yang didapatkan turun signifikan menjadi 0.5002.

Kemudian pada *uniform sampling*, kombinasi parameter terbaik diperoleh pada learning rate yang sama, konfigurasi ini menghasilkan Precision@10 sebesar 0.2871, Recall@10 sebesar 0.9961, NDCG@10 sebesar 0.8519, AUC sebesar 0.7624. Sementara itu, pada learning rate 0.01 hanya diperoleh Precision@10 sebesar 0.2876, Recall@10 sebesar 1.0000, NDCG@10 sebesar 8404, AUC sebesar 0.7221, dan pada learning rate 0.05 AUC yang didapatkan turun sangat signifikan menjadi 0.4997. Sehingga learning 0.001 dipilih sebagai konfigurasi terbaik karena memberikan performa evaluasi tertinggi pada data validasi.

Hasil tersebut akan dijadikan acuan untuk melanjutkan ke tahap pelatihan ulang model sepenuhnya pada data latih, agar model yang dihasilkan bisa belajar secara maksimal dari seluruh data yang tersedia.

4.5 Pelatihan Ulang

Setelah menemukan kombinasi parameter terbaik untuk masing-masing metode sampling, kedua model akan dilatih ulang kembali agar model yang dibangun optimal sebelum dilakukan pengujian.

Model yang menggunakan *importance sampling* dilatih ulang dengan learning rate 0.001 dan embedding size 32. Hasil terlihat pada gambar berikut.

```
=====Training Session=====
```

P@10: 0.5005	R@10: 1.0000	NDCG@10: 0.8745	AUC: 0.5479	Time: 100.07s	Total time: 100.07s
P@10: 0.4994	R@10: 1.0000	NDCG@10: 0.9495	AUC: 0.7712	Time: 105.07s	Total time: 205.14s
P@10: 0.4994	R@10: 1.0000	NDCG@10: 0.9660	AUC: 0.8451	Time: 102.64s	Total time: 307.78s
P@10: 0.4977	R@10: 0.9961	NDCG@10: 0.9641	AUC: 0.8591	Time: 100.77s	Total time: 408.54s
P@10: 0.5001	R@10: 1.0000	NDCG@10: 0.9687	AUC: 0.8642	Time: 107.37s	Total time: 515.91s
P@10: 0.4992	R@10: 1.0000	NDCG@10: 0.9684	AUC: 0.8669	Time: 109.17s	Total time: 625.08s
P@10: 0.4987	R@10: 1.0000	NDCG@10: 0.9696	AUC: 0.8696	Time: 114.39s	Total time: 739.47s
P@10: 0.5037	R@10: 1.0000	NDCG@10: 0.9712	AUC: 0.8711	Time: 107.05s	Total time: 846.52s
P@10: 0.5003	R@10: 0.9961	NDCG@10: 0.9676	AUC: 0.8730	Time: 107.38s	Total time: 953.89s
P@10: 0.4990	R@10: 1.0000	NDCG@10: 0.9715	AUC: 0.8758	Time: 103.90s	Total time: 1057.79s

Gambar IV. 7 Pelatihan Ulang Importance Sampling

Sementara itu, pada model yang menggunakan *uniform sampling* dilakukan pelatihan ulang juga dengan kombinasi yang sama yaitu learning rate 0.001 dan embedding size 32. Hasil terlihat pada gambar berikut.

```
=====Training Session=====
```

P@10: 0.2863	R@10: 1.0000	NDCG@10: 0.7662	AUC: 0.5228	Time: 91.94s	Total time: 91.94s
P@10: 0.2860	R@10: 1.0000	NDCG@10: 0.8346	AUC: 0.6588	Time: 91.42s	Total time: 183.36s
P@10: 0.2835	R@10: 0.9922	NDCG@10: 0.8748	AUC: 0.7633	Time: 91.34s	Total time: 274.70s
P@10: 0.2867	R@10: 1.0000	NDCG@10: 0.8914	AUC: 0.7946	Time: 90.97s	Total time: 365.67s
P@10: 0.2880	R@10: 1.0000	NDCG@10: 0.8939	AUC: 0.7995	Time: 90.02s	Total time: 455.70s
P@10: 0.2848	R@10: 0.9961	NDCG@10: 0.8907	AUC: 0.8021	Time: 89.94s	Total time: 545.63s
P@10: 0.2897	R@10: 1.0000	NDCG@10: 0.8974	AUC: 0.8042	Time: 89.68s	Total time: 635.32s
P@10: 0.2855	R@10: 1.0000	NDCG@10: 0.8973	AUC: 0.8043	Time: 90.73s	Total time: 726.05s
P@10: 0.2848	R@10: 0.9961	NDCG@10: 0.8939	AUC: 0.8054	Time: 94.60s	Total time: 820.65s
P@10: 0.2884	R@10: 1.0000	NDCG@10: 0.8969	AUC: 0.8070	Time: 106.78s	Total time: 927.43s

Gambar IV. 8 Pelatihan Uniform Sampling

Dari hasil pelatihan ulang, terlihat bahwa optimasi hyperparameter memberikan dampak positif pada kedua model. Model yang menerapkan *importance sampling* menunjukkan peningkatan performa yang cukup signifikan, dimana skor NDCG@10 dan AUC meningkat sekitar 1%. Di sisi lain, model dengan *uniform sampling* mengalami peningkatan yang sedikit lebih rendah, yaitu sekitar 0.85% pada NDCG@10 dan 0.82% pada AUC. Dari segi waktu training, kedua model mengalami waktu training lebih cepat, waktu per epoch untuk model yang menerapkan *importance sampling* turun dari rata-rata 566 detik menjadi 106 detik, sedangkan pada model yang menerapkan *uniform sampling* turun dari rata-rata 537 detik menjadi sekitar 93 detik. Hal ini membuktikan bahwa ukuran

embedding size 32 dapat menurunkan waktu training tanpa harus mengorbankan akurasi.

4.6 Test Model

Kemudian dilakukan pengujian model pada data tes, yang merupakan data yang belum pernah dilihat oleh model. Pengujian ini dilakukan pada kedua model (*importance sampling* dan *uniform sampling*) yang telah dilatih ulang menggunakan hyperparameter terbaik. Hasil pengujian akhir dari kedua model dapat dilihat pada tabel berikut:

Tabel IV. 5 Hasil Evaluasi Data Tes

Metrik	Importance Sampling	Uniform Sampling
AUC	0.8798	0.8147
NDCG@10	0.9719	0.8983
Precision@10	0.4988	0.2845
Recall@10	1.0000	0.9961

Berdasarkan tabel perbandingan di atas, terlihat jelas bahwa model yang menggunakan *importance sampling* secara konsisten mengungguli model dengan *uniform sampling* di semua metrik evaluasi. Dari metrik Precision@10 teknik *importance sampling* menunjukkan kemampuan yang signifikan dalam merekomendasikan item yang relevan kepada user. Hal ini dibuktikan dengan skor yang mencapai 0.4988 pada *importance sampling* dibandingkan dengan *uniform sampling*.

4.7 Analisis dan Studi Kasus: Perbandingan Pengguna Baru vs Pengguna Lama

Dilakukan analisis kualitatif untuk memahami perilaku model KGCN dengan *importance sampling* dalam memberikan rekomendasi. Analisis ini bertujuan untuk melihat contoh hasil rekomendasi pada pengguna spesifik dan menginterpretasi relevansi berdasarkan data demografis pada *knowledge graph*.

4.7.1 Studi Kasus Pengguna Baru

Studi kasus dilakukan pada pengguna baru bernama Poppy. Berdasarkan data demografis, pengguna ini adalah seorang wanita (*female*), dalam kelompok usia 18-24 tahun, dan seorang mahasiswa. Profil ini menunjukkan seorang remaja yang

preferensinya dapat diidentifikasi oleh model. Gambar IV.9 menunjukkan hasil Top-10 film yang direkomendasikan oleh model untuk pengguna tersebut:

Top 10 Rekomendasi

Judul Film	Genre
Saving Private Ryan (1998)	Action, Drama, War
Braveheart (1995)	Action, Drama, War
Terminator 2: Judgment Day (1991)	Action, Sci-Fi, Thriller
Matrix, The (1999)	Action, Sci-Fi, Thriller
Aliens (1986)	Action, Sci-Fi, Thriller, War
Terminator, The (1984)	Action, Sci-Fi, Thriller
Fugitive, The (1993)	Action, Thriller
Star Wars: Episode V - The Empire Strikes Back (1980)	Action, Adventure, Drama, Sci-Fi, War
Star Wars: Episode VI - Return of the Jedi (1983)	Action, Adventure, Romance, Sci-Fi, War
Silence of the Lambs, The (1991)	Drama, Thriller

Gambar IV. 9 Hasil Rekomendasi Pengguna Baru

Studi kasus ini mengilustrasikan keunggulan utama model dalam menangani skenario *cold-start*. Keberhasilan ini tidak hanya bertumpu pada kekuatan *Knowledge Graph* dalam memetakan kesamaan demografis, tetapi juga diperkuat oleh teknik *Importance Sampling*. Selama mekanisme *message passing*, *Importance Sampling* secara cerdas memilih pengguna lama yang paling representatif dari klaster demografis yang sama. Proses selektif ini memastikan bahwa pengguna baru dikelompokkan dalam lingkungan yang paling relevan, sehingga kemampuan model untuk mitigasi masalah cold-start menjadi lebih efektif.

4.7.2 Studi Kasus: Pengguna Lama

Studi kasus dilakukan pada pengguna lama dengan UserID 204. Berdasarkan data demografis pengguna ini adalah seorang pria (*Male*), berada dikelompok usia 25-34 tahun, dan bekerja sebagai seorang programmer. Berikut hasil rekomendasi untuk pengguna lama disajikan pada Gambar IV.10.

Berikut rekomendasi khusus untuk pengguna 204.

Top 10 Rekomendasi

Judul Film	Genre
Saving Private Ryan (1998)	Action, Drama, War
Silence of the Lambs, The (1991)	Drama, Thriller
Star Wars: Episode V - The Empire Strikes Back (1980)	Action, Adventure, Drama, Sci-Fi, War
Braveheart (1995)	Action, Drama, War
Fargo (1996)	Crime, Drama, Thriller
2001: A Space Odyssey (1968)	Drama, Mystery, Sci-Fi, Thriller
Schindler's List (1993)	Drama, War
Talented Mr. Ripley, The (1999)	Drama, Mystery, Thriller
Casablanca (1942)	Drama, Romance, War
Aliens (1986)	Action, Sci-Fi, Thriller, War

Gambar IV. 10 Hasil Rekomendasi Pengguna Lama

Studi kasus pada Pengguna 204 ini menjelaskan bahwa model mampu memberikan rekomendasi yang sangat personal kepada pengguna lama dengan menganalisis secara mendalam riwayat rating yang telah diberikan. Kemampuan personalisasi ini terlihat dari keberhasilan model dalam mengidentifikasi preferensi genre yang spesifik. Rekomendasi yang diberikan tidak terbatas pada satu genre Tunggal, melainkan pada kombinasi beberapa genre yang memiliki cerita mendalam, seperti kombinasi *Drama*, *Thriller*, dan *Sci-Fi*. Peran teknik *importance sampling* juga menjadi kunci dalam keberhasilan ini, karena pengguna 204 memiliki riwayat rating yang telah diberikan, *importance sampling* tidak memilih tetangga secara acak melainkan memprioritaskan *node* tetangga yang memiliki hubungan kuat dengan genre tersebut. Proses selektif ini memastikan bahwa agregasi informasi menjadi lebih fokus dan menghasilkan representasi *embedding* yang secara presisi mencerminkan selera pengguna, yang akhirnya menghasilkan daftar rekomendasi yang relevan dan personalisasi.

BAB V KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan hasil analisis dan eksperimen, penelitian ini menyimpulkan bahwa penerapan teknik importance sampling secara konsisten menghasilkan model rekomendasi yang lebih unggul dibandingkan uniform sampling pada seluruh metrik evaluasi. Ditemukan adanya hal yang perlu dikorbankan antara akurasi dan efisiensi, di mana importance sampling memerlukan waktu komputasi per epoch yang sedikit lebih lama. Namun, hal ini dapat dibenarkan oleh peningkatan akurasi yang sangat signifikan. Penambahan data pengguna (user-end) ke dalam Knowledge Graph terbukti berhasil memperkaya pemahaman model terhadap preferensi pengguna secara personal. Selain itu, proses hyperparameter tuning menunjukkan bahwa arsitektur model yang lebih ramping (embedding size 32) tidak hanya meningkatkan akurasi tetapi juga efisiensi waktu komputasi secara drastis. Hasil pengujian akhir yang solid pada data tes mengonfirmasi bahwa model yang dioptimalkan memiliki kemampuan generalisasi yang baik untuk data baru. Secara keseluruhan, penelitian ini membuktikan bahwa pendekatan KGCN yang diintegrasikan dengan importance sampling merupakan strategi yang efektif dan andal untuk membangun sistem rekomendasi film yang akurat.

5.2 Saran

Saran untuk pengembangan lebih lanjut dari penelitian ini

1. Penelitian dapat diperluas dengan menambahkan atribut tambahan seperti waktu, lokasi atau *mood* pengguna agar model rekomendasi menjadi lebih kontekstual dan personal.
2. Penggunaan *Relational Graph Convolutional Network* (R-GCN) atau *Graph Attention Network* (GAT) dapat menjadi alternatif yang menarik untuk menangkap relasi multi-tipe antar node dengan lebih akurat.
3. Menambahkan implementasi dropout pada model GCN untuk mengatasi masalah *overfitting* yang terjadi, dimana teknik regularisasi ini dapat

menjadi fokus utama untuk meningkatkan kemampuan adaptasi model terhadap data baru.

4. Untuk menguji kemampuan generalisasi model pada domain yang berbeda, penelitian selanjutnya dapat melakukan evaluasi menggunakan dataset lain. Dataset seperti Amazon Review, Netflix Prize, atau Book-Crossing memiliki karakteristik pengguna dan item yang berbeda, sehingga dapat memberikan wawasan baru mengenai seberapa andal (*robust*) pendekatan yang diusulkan.

DAFTAR PUSTAKA

- [1] W. Wołowczyk and E. Szymik, “Movie Recommendation System,” in *CEUR Workshop Proceedings*, CEUR-WS, 2024, pp. 406–414. doi: 10.48175/ijarsct-17453.
- [2] A. Karande, A. Salunke, and C. Shetty, “Movie Recommendation System.” [Online]. Available: <https://ssrn.com/abstract=4846260>
- [3] Q. Guo *et al.*, “A Survey on Knowledge Graph-Based Recommender Systems,” *IEEE Trans Knowl Data Eng*, vol. 34, no. 8, pp. 3549–3568, Aug. 2022, doi: 10.1109/TKDE.2020.3028705.
- [4] H. Wang, M. Zhao, X. Xie, W. Li, and M. Guo, “Knowledge graph convolutional networks for recommender systems,” in *The Web Conference 2019 - Proceedings of the World Wide Web Conference, WWW 2019*, Association for Computing Machinery, Inc, May 2019, pp. 3307–3313. doi: 10.1145/3308558.3313417.
- [5] Y. Ji *et al.*, “Accelerating Large-Scale Heterogeneous Interaction Graph Embedding Learning via Importance Sampling,” *ACM Trans Knowl Discov Data*, vol. 15, no. 1, Jan. 2021, doi: 10.1145/3418684.
- [6] Y. Afoudi, M. Lazaar, and M. Al Achhab, “Hybrid recommendation system combined content-based filtering and collaborative prediction using artificial neural network,” *Simul Model Pract Theory*, vol. 113, Dec. 2021, doi: 10.1016/j.simpat.2021.102375.
- [7] T. Gu, H. Liang, C. Bin, and L. Chang, “Combining user-end and item-end knowledge graph learning for personalized recommendation,” *Journal of Intelligent and Fuzzy Systems*, vol. 40, no. 5, pp. 9213–9225, 2021, doi: 10.3233/JIFS-201635.
- [8] A. Hicham, P. A. Sabri, and P. H. Tairi, “A Survey on Educational Data Mining [2014-2019] 1 st,” 2020.
- [9] W. T. Wu *et al.*, “Data mining in clinical big data: the frequently used databases, steps, and methodological models,” Dec. 01, 2021, *BioMed Central Ltd*. doi: 10.1186/s40779-021-00338-z.

- [10] A. Bartschat, M. Reischl, and R. Mikut, “Data mining tools,” *Wiley Interdiscip Rev Data Min Knowl Discov*, vol. 9, no. 4, Jul. 2019, doi: 10.1002/widm.1309.
- [11] Z. Batmaz, A. Yurekli, A. Bilge, and C. Kaleli, “A review on deep learning for recommender systems: challenges and remedies,” *Artif Intell Rev*, vol. 52, no. 1, pp. 1–37, Jun. 2019, doi: 10.1007/s10462-018-9654-y.
- [12] H. Ko, S. Lee, Y. Park, and A. Choi, “A Survey of Recommendation Systems: Recommendation Models, Techniques, and Application Fields,” Jan. 01, 2022, *MDPI*. doi: 10.3390/electronics11010141.
- [13] F. O. Isinkaye, Y. O. Folajimi, and B. A. Ojokoh, “Recommendation systems: Principles, methods and evaluation,” Nov. 01, 2015, *Elsevier B.V.* doi: 10.1016/j.eij.2015.06.005.
- [14] R. Widayanti, M. Heru, R. Chakim, C. Lukita, U. Rahardja, and N. Lutfiani, “Improving Recommender Systems using Hybrid Techniques of Collaborative Filtering and Content-Based Filtering,” *Journal of Applied Data Sciences*, vol. 4, no. 3, pp. 289–302, 2023.
- [15] L. Wang and Y. Zhang, “Digital Dissemination of Information Based on Knowledge Graph Recommendation Algorithm,” *Informatica (Slovenia)*, vol. 48, no. 19, pp. 31–44, 2024, doi: 10.31449/inf.v48i19.6561.
- [16] J. A. P. Sacenti, R. Fileto, and R. Willrich, “Knowledge graph summarization impacts on movie recommendations,” *J Intell Inf Syst*, vol. 58, no. 1, pp. 43–66, Feb. 2022, doi: 10.1007/s10844-021-00650-z.
- [17] X. Wang *et al.*, “Learning intents behind interactions with knowledge graph for recommendation,” in *The Web Conference 2021 - Proceedings of the World Wide Web Conference, WWW 2021*, Association for Computing Machinery, Inc, Apr. 2021, pp. 878–887. doi: 10.1145/3442381.3450133.
- [18] H. Xia, K. Huang, and Y. Liu, “Unexpected interest recommender system with graph neural network,” *Complex and Intelligent Systems*, vol. 9, no. 4, pp. 3819–3833, Aug. 2023, doi: 10.1007/s40747-022-00849-9.
- [19] Y. Zheng, C. Gao, L. Chen, D. Jin, and Y. Li, “DGCN: Diversified recommendation with graph convolutional networks,” in *The Web*

- Conference 2021 - Proceedings of the World Wide Web Conference, WWW 2021*, Association for Computing Machinery, Inc, Jun. 2021, pp. 401–412. doi: 10.1145/3442381.3449835.
- [20] Y. Wang, X. F. Ma, and M. Zhu, “A knowledge graph algorithm enabled deep recommendation system,” *PeerJ Comput Sci*, vol. 10, 2024, doi: 10.7717/PEERJ-CS.2010.
 - [21] U. A. Bhatti, H. Tang, G. Wu, S. Marjan, and A. Hussain, “Deep Learning with Graph Convolutional Networks: An Overview and Latest Applications in Computational Intelligence,” 2023, *Wiley-Hindawi*. doi: 10.1155/2023/8342104.
 - [22] C. Li, Y. Cao, Y. Zhu, D. Cheng, C. Li, and Y. Morimoto, “Ripple Knowledge Graph Convolutional Networks for Recommendation Systems,” *Machine Intelligence Research*, vol. 21, no. 3, pp. 481–494, Jun. 2024, doi: 10.1007/s11633-023-1440-x.
 - [23] J. Chen, T. Ma, and C. Xiao, “FastGCN: Fast Learning with Graph Convolutional Networks via Importance Sampling,” Jan. 2018, [Online]. Available: <http://arxiv.org/abs/1801.10247>
 - [24] S. Nur Himawa and R. Sohiburoyyan, “Hyperparameter Tuning on Graph Neural Network for the Classification of SARS-CoV-2 Inhibitors,” 2023. [Online]. Available: <http://jurnal.polibatam.ac.id/index.php/JAIC>
 - [25] X. Zeng, S. Wang, Y. Zhu, M. Xu, and Z. Zou, “A Knowledge Graph Convolutional Networks Method for Countryside Ecological Patterns Recommendation by Mining Geographical Features,” *ISPRS Int J Geoinf*, vol. 11, no. 12, Dec. 2022, doi: 10.3390/ijgi11120625.
 - [26] O. Rainio, J. Teuho, and R. Klén, “Evaluation metrics and statistical tests for machine learning,” *Sci Rep*, vol. 14, no. 1, Dec. 2024, doi: 10.1038/s41598-024-56706-x.
 - [27] S. Siet, S. Peng, S. Ilkhomjon, M. Kang, and D. S. Park, “Enhancing Sequence Movie Recommendation System Using Deep Learning and KMeans,” *Applied Sciences (Switzerland)*, vol. 14, no. 6, Mar. 2024, doi: 10.3390/app14062505.

- [28] S. Subhan, D. L. Syarif, E. Widhihastuti, S. K. Rakainsa, M. Sam'an, and Y. N. Ifriza, "Improved recommender system using Neural Network Collaborative Filtering (NNCF) for E-commerce cosmetic product," *Sinergi (Indonesia)*, vol. 29, no. 1, pp. 155–162, 2025, doi: 10.22441/sinergi.2025.1.014.
- [29] H. Yang, J. Li, and S. Jahng, "The Application of Knowledge Graph Convolutional Network-based Film and Television Interaction under Artificial Intelligence," *IEEE Access*, 2024, doi: 10.1109/ACCESS.2024.3459448.
- [30] T. Al-shehari and R. A. Alsowail, "An insider data leakage detection using one-hot encoding, synthetic minority oversampling and machine learning techniques," *Entropy*, vol. 23, no. 10, Oct. 2021, doi: 10.3390/e23101258.
- [31] Intan Permata and Esther Sorta Mauli Nababan, "Application Of Game Theory In Determining Optimum Marketing Strategy In Marketplace," *JURNAL RISET RUMPUN MATEMATIKA DAN ILMU PENGETAHUAN ALAM*, vol. 2, no. 2, pp. 65–71, Jul. 2023, doi: 10.55606/jurrimipa.v2i2.1336.

LAMPIRAN

Penggabungan Dataset

```
all_df = pd.merge(left=users, right=ratings, on='UserID', how='inner')
all_df = pd.merge(left=all_df, right=movies, on='MovieID', how='inner')
all_df.info()
```

Label Encoding

```
#Inisialisasi variable
le = LabelEncoder()
le.fit(all_df['Gender']) #Simpan urutan encoding
all_df['Gender'] = le.fit_transform(all_df['Gender'])
```

Knowledge Graph

Mapping ID

```
# Mapping ID unik
user_mapping = {id_: i for i, id_ in enumerate(all_df['UserID'].unique())}
movie_mapping = {id_: i for i, id_ in enumerate(all_df['MovieID'].unique())}
gender_mapping = {g: i for i, g in enumerate(all_df['Gender'].unique())}
age_mapping = {a: i for i, a in enumerate(all_df['Age'].unique())}
occupation_mapping = {o: i for i, o in enumerate(all_df['Occupation'].unique())}

# Mengumpulkan genre unik dan buat mapping genre menjadi indeks angka
genres = set()
for genre_list in all_df['Genres']:
    genres.update(genre_list)
genre_mapping = {g: i for i, g in enumerate(sorted(genres))}
```

Pembuatan *Node*

```
#Representasi Node
num_users = len(user_mapping)
```



```

num_movies = len(movie_mapping)
num_genders = len(gender_mapping)
num_ages = len(age_mapping)
num_occupations = len(occupation_mapping)
num_genres = len(genre_mapping)

print(f'Users: {num_users}, Movies: {num_movies}, Genders: {num_genders},
Ages: {num_ages}, Occupations: {num_occupations}, Genres: {num_genres}')

```

Relasi Antar *Node*

```

# user → movie edge
user_movie_edge = [
    [user_mapping[row['UserID']], movie_mapping[row['MovieID']]]
    for _, row in all_df.iterrows()
]
user_movie_edge = torch.tensor(user_movie_edge).t().contiguous()
edge_weight = torch.tensor(all_df['Rating'].values, dtype=torch.float)

# user → gender edge
user_gender_edge = [
    [user_mapping[row['UserID']], gender_mapping[row['Gender']]]
    for _, row in all_df.iterrows()
]
user_gender_edge = torch.tensor(user_gender_edge).t().contiguous()

# user → age edge
user_age_edge = [
    [user_mapping[row['UserID']], age_mapping[row['Age']]]
    for _, row in all_df.iterrows()
]

```

```

user_age_edge = torch.tensor(user_age_edge).t().contiguous()

# user → occupation edge
user_occupation_edge = [
    [user_mapping[row['UserID']], occupation_mapping[row['Occupation']]]
    for _, row in all_df.iterrows()
]
user_occupation_edge = torch.tensor(user_occupation_edge).t().contiguous()

# movie → genre edge
movie_genre_edge = []
for _, row in all_df.iterrows():
    movie_id = movie_mapping[row['MovieID']]
    for genre in row['Genres']:
        genre_id = genre_mapping[genre]
        movie_genre_edge.append([movie_id, genre_id])
movie_genre_edge = torch.tensor(movie_genre_edge).t().contiguous()

```

Triplet Hubungan Antar *Node*

```

data = HeteroData()

# Node counts
data['user'].num_nodes = num_users
data['movie'].num_nodes = num_movies
data['gender'].num_nodes = num_genders
data['age'].num_nodes = num_ages
data['occupation'].num_nodes = num_occupations
data['genre'].num_nodes = num_genres

# Edge connections

```

```

data['user', 'rates', 'movie'].edge_index = user_movie_edge
data['user', 'rates', 'movie'].edge_weight = edge_weight
data['user', 'has_gender', 'gender'].edge_index = user_gender_edge
data['user', 'has_age', 'age'].edge_index = user_age_edge
data['user', 'has_occupation', 'occupation'].edge_index = user_occupation_edge
data['movie', 'has_genre', 'genre'].edge_index = movie_genre_edge

for ntype in data.node_types:
    data[ntype].node_id = torch.arange(data[ntype].num_nodes)

#Reverse edges untuk pass message
data = T.ToUndirected()(data)

print(data.edge_types)

```

Splitting Dataset

```

transform = T.RandomLinkSplit(
    num_val=0.10, #10% data validation
    num_test=0.10, #10% data testing
    add_negative_train_samples=False,
    neg_sampling_ratio=0.0,
    edge_types=('user', 'rates', 'movie'),
    rev_edge_types=('movie', 'rev_rates', 'user')
)

train_data, val_data, test_data = transform(data)
total_edges = data['user', 'rates', 'movie'].edge_index.size(1)

print(f'Total Edges: {total_edges}')

```

```

print("Training data: ", train_data['user', 'rates',
'movie'].edge_label_index.size(1))
print("Validation data: ", val_data['user', 'rates',
'movie'].edge_label_index.size(1))
print("Testing data: ", test_data['user', 'rates', 'movie'].edge_label_index.size(1))

```

Importance Sampling

```

def get_importance_sampled_loader(train_data, threshold=4.0,
max_samples=100000):
    # Ambil edge dan rating
    edge_index = train_data["user", "rates", "movie"].edge_index
    edge_weight = train_data["user", "rates", "movie"].edge_weight.float()

    # Filter rating >= threshold
    mask = edge_weight >= threshold
    filtered_edge_index = edge_index[:, mask]
    filtered_edge_weight = edge_weight[mask]

    # Normalisasi sebagai probabilitas sampling
    prob = filtered_edge_weight / filtered_edge_weight.sum()

    # Importance sampling
    sample_size = min(max_samples, filtered_edge_weight.size(0))
    sampled_idx = torch.multinomial(prob, num_samples=sample_size,
replacement=False)

    sampled_edge_index = filtered_edge_index[:, sampled_idx]
    sampled_edge_weight = filtered_edge_weight[sampled_idx]

    # Loader

```

```

loader = LinkNeighborLoader(
    data=train_data,
    num_neighbors={edge_type: [5, 5] for edge_type in
train_data.edge_types},
    edge_label_index=(("user", "rates", "movie"), sampled_edge_index),
    edge_label=sampled_edge_weight,
    batch_size=128,
    shuffle=True,
    neg_sampling_ratio=1.0
)

return loader, sampled_edge_index, sampled_edge_weight

```

Uniform Sampling

```

def get_uniform_sampled_loader(train_data, num_samples):
    edge_index = train_data["user", "rates", "movie"].edge_index
    edge_weight = train_data["user", "rates", "movie"].edge_weight.float()

    uniform_idx = torch.randperm(edge_index.size(1))[:num_samples]
    sampled_edge_index_uniform = edge_index[:, uniform_idx]
    sampled_edge_weight_uniform = edge_weight[uniform_idx]

    loader = LinkNeighborLoader(
        data=train_data,
        num_neighbors={edge_type: [5, 5] for edge_type in
train_data.edge_types},
        edge_label_index=(("user", "rates", "movie"),
sampled_edge_index_uniform),
        edge_label=sampled_edge_weight_uniform,
        batch_size=128,

```

```

        shuffle=True,
        neg_sampling_ratio=1.0
    )
    return loader, sampled_edge_weight_uniform

```

Graph Convolutional Network

```

class GCN(torch.nn.Module):
    def __init__(self, hidden_channels):
        super().__init__()
        self.conv1 = SAGEConv(hidden_channels, hidden_channels, aggr='sum')
        self.conv2 = SAGEConv(hidden_channels, hidden_channels, aggr='sum')

    def forward(self, x: Tensor, edge_index: Tensor) -> Tensor:
        x = self.conv1(x, edge_index)
        x = F.relu(x)
        x = self.conv2(x, edge_index)
        return x

class Classifier(torch.nn.Module):
    def forward(self, x_user, x_movie, edge_label_index):
        user_feat = x_user[edge_label_index[0]]
        movie_feat = x_movie[edge_label_index[1]]
        return (user_feat * movie_feat).sum(dim=-1)

class Model(torch.nn.Module):
    def __init__(self, hidden_channels, metadata, num_nodes_dict):
        super().__init__()

        # Embedding Node
        self.embeddings = nn.ModuleDict({

```

```

        ntype: nn.Embedding(num_nodes_dict[ntype], hidden_channels)
        for ntype in metadata[0]
    })
    self.GCN = GCN(hidden_channels)
    self.GCN = to_hetero(self.GCN, metadata=metadata)

    # Classifier prediksi dot-product antar user -> movie
    self.classifier = Classifier()

def forward(self, data, edge_label_index=None):
    x_dict = {
        ntype: self.embeddings[ntype](data[ntype].node_id)
        for ntype in data.node_types
    }

    x_dict = self.GCN(x_dict, data.edge_index_dict)

    if edge_label_index is not None:
        return self.classifier(
            x_dict['user'],
            x_dict['movie'],
            edge_label_index
        )
    else:
        return x_dict

```

Evaluasi Model

```

def evaluate_ranking(pred_scores, edge_index, edge_label, k=10):
    """
    pred_scores : Tensor hasil prediksi

```

```

    edge_index : Tensor indeks [2, num_edges], baris 0 = user_id, baris 1 =
movie_id
    edge_label : Tensor label rating asli
    k          : Jumlah rekomendasi teratas yang dipertimbangkan
    """
    # Label relevan dengan threshold 4 dianggap relevan
    relevance = (edge_label >= 4).float()

    # Ambil UserID sebagai index
    user_ids = edge_index[0]

    # Inisialisasi metrik
    precision = RetrievalPrecision()
    recall = RetrievalRecall()
    ndcg = RetrievalNormalizedDCG()

    # Update metrik dengan k
    precision.update(pred_scores, relevance, indexes=user_ids, top_k=k)
    recall.update(pred_scores, relevance, indexes=user_ids, top_k=k)
    ndcg.update(pred_scores, relevance, indexes=user_ids, top_k=k)

    return {
        f'Precision@{k}': precision.compute().item(),
        f'Recall@{k}': recall.compute().item(),
        f'NDCG@{k}': ndcg.compute().item()
    }

def train(model, loader, loss_fn, device, optimizer, max_epoch, k,
sampled_edge_weight, epoch_desc='Training'):
    historyTrain = {
        f'precision@{k}': [],

```



```

frecall@{k}': [],
fndcg@{k}': [],
fauc': [],
'epoch_time': [],
}

print("====Training Session====")

# Hitung distribusi sampling untuk importance weight
eps = 1e-8
q_dist = sampled_edge_weight / sampled_edge_weight.sum()
p_dist = torch.softmax(sampled_edge_weight, dim=0)
importance_weight_full = (p_dist / (q_dist + eps)).detach()

for epoch in range(max_epoch):
    model.train()
    total_loss = 0.0
    total_samples = 0

    # Init metrik
    precision = RetrievalPrecision().to(device)
    recall = RetrievalRecall().to(device)
    ndcg = RetrievalNormalizedDCG().to(device)
    auc = BinaryAUROC().to(device)

    epoch_start = time.time()
    progress_bar = tqdm.tqdm(loader, desc=f'{epoch_desc} Epoch
{epoch:03d}', leave=False)

    for batch_id, batch in enumerate(progress_bar):
        batch = batch.to(device)

```

```

optimizer.zero_grad()

label = batch['user', 'rates', 'movie'].edge_label.float()
edge_index = batch['user', 'rates', 'movie'].edge_label_index
pred = model(batch, edge_index)

# Ambil importance weight yang sesuai batch
edge_ids_in_batch = batch['user', 'rates', 'movie'].edge_label_index[0]
importance_weight = importance_weight_full[edge_ids_in_batch]

# Hitung MSE per sample
mse_loss_each = F.mse_loss(pred, label, reduction='none')
weighted_loss = (importance_weight * mse_loss_each).mean()

weighted_loss.backward()
clip_grad_norm_(model.parameters(), max_norm=1.0)
optimizer.step()

# Logging
batch_size = label.size(0)
total_loss += weighted_loss.item() * batch_size
total_samples += batch_size

# Evaluasi metrik
relevance = (label >= 4).float()
precision.update(pred, relevance, indexes=edge_index[0])
recall.update(pred, relevance, indexes=edge_index[0])
ndcg.update(pred, relevance, indexes=edge_index[0])
auc.update(pred, relevance)

# Epoch selesai

```

```

epoch_duration = time.time() - epoch_start
historyTrain['epoch_time'].append(epoch_duration)
historyTrain[f'precision@{k}'].append(precision.compute().item())
historyTrain[f'recall@{k}'].append(recall.compute().item())
historyTrain[f'ndcg@{k}'].append(ndcg.compute().item())
historyTrain[f'auc'].append(auc.compute().item())

```

```

print(f'P@{k}: {historyTrain[f'precision@{k}'][-1]:.4f} | "
      f'R@{k}: {historyTrain[f'recall@{k}'][-1]:.4f} | "
      f'NDCG@{k}: {historyTrain[f'ndcg@{k}'][-1]:.4f} | "
      f'AUC: {historyTrain[f'auc'][-1]:.4f} | "
      f'Time: {epoch_duration:.2f}s | "
      f'Total time: {sum(historyTrain['epoch_time']):.2f}s")

```

```

return historyTrain

```

```

def train_with_hyperparams(model, train_loader, val_loader, loss_fn, device,
optimizer, max_epoch, k, sampled_edge_weight, epoch_desc='Training',
patience=5):

```

```

    historyVal = {
        f'precision@{k}': [],
        f'recall@{k}': [],
        f'ndcg@{k}': [],
        'auc': []
    }

```

```

    # Importance sampling: hitung distribusi sampling penuh

```

```

    eps = 1e-8
    q_dist = sampled_edge_weight / sampled_edge_weight.sum()
    p_dist = torch.softmax(sampled_edge_weight, dim=0)
    importance_weight_full = (p_dist / (q_dist + eps)).detach()

```

```

early_stopper = EarlyStopping(patience=patience, mode='max',
verbose=True)

for epoch in range(max_epoch):
    model.train()

    for batch in tqdm.tqdm(train_loader, desc=f" {epoch_desc} Epoch
{epoch:03d}", leave=False):
        batch = batch.to(device)
        optimizer.zero_grad()

        edge_label_index = batch["user", "rates", "movie"].edge_label_index
        label = batch["user", "rates", "movie"].edge_label.float()
        pred = model(batch, edge_label_index=edge_label_index)

        # Ambil importance weight dari edge yang digunakan di batch ini
        edge_ids_in_batch = edge_label_index[0]
        importance_weight = importance_weight_full[edge_ids_in_batch]

        # Hitung weighted loss dengan importance sampling
        mse_loss_each = F.mse_loss(pred, label, reduction='none')
        weighted_loss = (importance_weight * mse_loss_each).mean()

        weighted_loss.backward()
        clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()

    # Validation setiap epoch
    val_metrics = evaluate(val_loader, model, device, k)
    for key in historyVal:

```

```

        historyVal[key].append(val_metrics[key])

    early_stopper.on_epoch_end(val_metrics['auc'])

    print(
        f"Epoch {epoch} | "
        f"Val Precision@{k}: {val_metrics[f'precision@{k}']:.4f}, "
        f"Recall@{k}: {val_metrics[f'recall@{k}']:.4f}, "
        f"NDCG@{k}: {val_metrics[f'ndcg@{k}']:.4f}, "
        f"AUC: {val_metrics['auc']:.4f}"
    )

    if early_stopper.early_stop:
        print(">> Early Stopping triggered!")
        break
    return historyVal

def evaluate(loader, model, device, k=10):
    model.eval()

    precision = RetrievalPrecision().to(device)
    recall = RetrievalRecall().to(device)
    ndcg = RetrievalNormalizedDCG().to(device)
    auc = BinaryAUROC().to(device)

    all_pred = []
    all_labels = []

    with torch.no_grad():
        for batch in tqdm.tqdm(loader, desc="Evaluating", leave=False):
            batch = batch.to(device)

```

```

edge_index = batch['user', 'rates', 'movie'].edge_label_index
pred = model(batch, edge_label_index=edge_index)

label = batch['user', 'rates', 'movie'].edge_label.float()

relevance = (label >= 4).float()

all_pred.append(pred.detach().cpu())
all_labels.append(relevance.detach().cpu())

precision.update(pred, relevance, indexes=edge_index[0])
recall.update(pred, relevance, indexes=edge_index[0])
ndcg.update(pred, relevance, indexes=edge_index[0])
auc.update(pred, relevance)

result = {
    f'precision@{k}': precision.compute().item(),
    f'recall@{k}': recall.compute().item(),
    f'ndcg@{k}': ndcg.compute().item(),
    'auc': auc.compute().item()
}

print(
    f'Precision@{k}: {result[f'precision@{k}']:4f} | '
    f'Recall@{k}: {result[f'recall@{k}']:4f} | '
    f'NDCG@{k}: {result[f'ndcg@{k}']:4f} | '
    f'AUC: {result['auc']:4f}"
)
return result

```